

本节内容

顺序查找

知识总览

顺序查找

算法思想

算法实现

算法优化

顺序查找的算法思想

顺序查找，又叫“线性查找”，通常用于线性表。

算法思想：从头到 jio 挨个找（或者反过来也OK）

33	10	13	29	16	19	32	7	43	41	37
----	----	----	----	----	----	----	---	----	----	----



查找目标：

43

顺序查找的算法思想



顺序查找，又叫“线性查找”，通常用于线性表。

算法思想：从头到 jio 挨个找（或者反过来也OK）



查找目标：

43

顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```

查找目标:

43

33	10	13	29	16	19	32	7	43	41	37				
----	----	----	----	----	----	----	---	----	----	----	--	--	--	--

0

1

2

3

4

5

6

7

8

9

10

11

12

13

...

TableLen=11

顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```

查找目标:

43

33	10	13	29	16	19	32	7	43	41	37				
----	----	----	----	----	----	----	---	----	----	----	--	--	--	--

0 1 2 3 4 5 6 7 8 9 10 11 12 13 ...

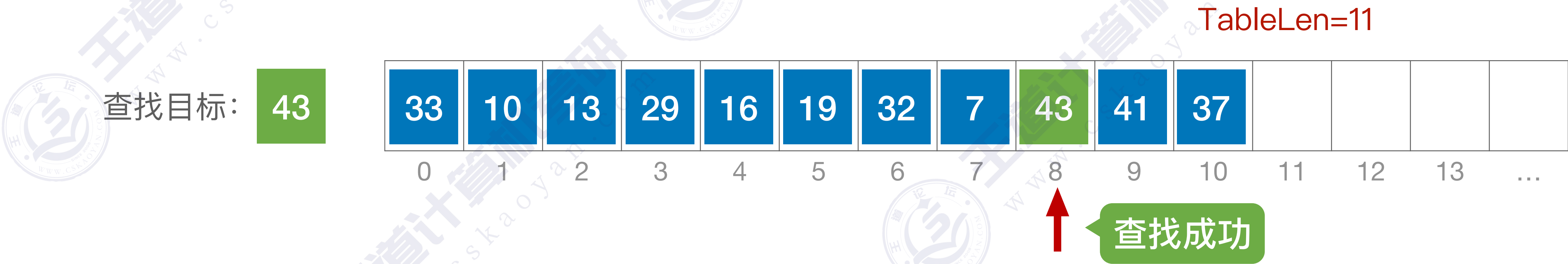


TableLen=11

顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```



顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```

查找目标:

66

33	10	13	29	16	19	32	7	43	41	37				
----	----	----	----	----	----	----	---	----	----	----	--	--	--	--

0 1 2 3 4 5 6 7 8 9 10 11 12 13 ...



TableLen=11

顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```

查找目标:

66

33	10	13	29	16	19	32	7	43	41	37				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

TableLen=11

↑ 查找失败

顺序查找的实现（哨兵）

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;
```

```
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;       //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;             //查找成功，则返回元素下标；查找失败，则返回0
}
```

0号位置存“哨兵”



数据从下标1开始存

16	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

TableLen=11



顺序查找的实现（哨兵）

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;
```

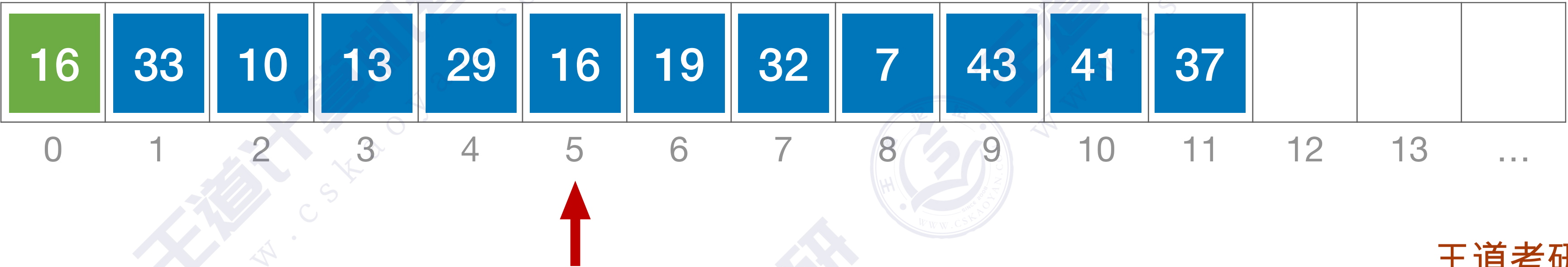
```
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;       //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;             //查找成功，则返回元素下标；查找失败，则返回0
}
```

0号位置存“哨兵”



数据从下标1开始存

TableLen=11



顺序查找的实现（哨兵）

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;
```

```
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;       //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;             //查找成功，则返回元素下标；查找失败，则返回0
}
```

0号位置存“哨兵”



数据从下标1开始存

TableLen=11

16	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...



查找成功

顺序查找的实现（哨兵）

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;
```

```
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;       //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;             //查找成功，则返回元素下标；查找失败，则返回0
}
```

0号位置存“哨兵”



数据从下标1开始存

66	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

TableLen=11



顺序查找的实现（哨兵）

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;
```

```
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;       //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;             //查找成功，则返回元素下标；查找失败，则返回0
}
```

优点：无需判断是否越界，效率更高

0号位置存“哨兵”



数据从下标1开始存

TableLen=11

66	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...



查找失败

查找效率分析



16	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

TableLen=11



查找成功

$$ASL = \sum_{i=1}^n P_i C_i$$

$$ASL_{\text{成功}} = \frac{1 + 2 + 3 + \dots + n}{n} = \frac{n + 1}{2}$$

$$ASL_{\text{失败}} = n + 1$$

TableLen=11

66	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...



查找失败

顺序查找的优化（对有序表）

查找表中元素有序存放（递增/递减）

查找目标：

21

7

13

19

29

37

43

顺序查找的优化（对有序表）

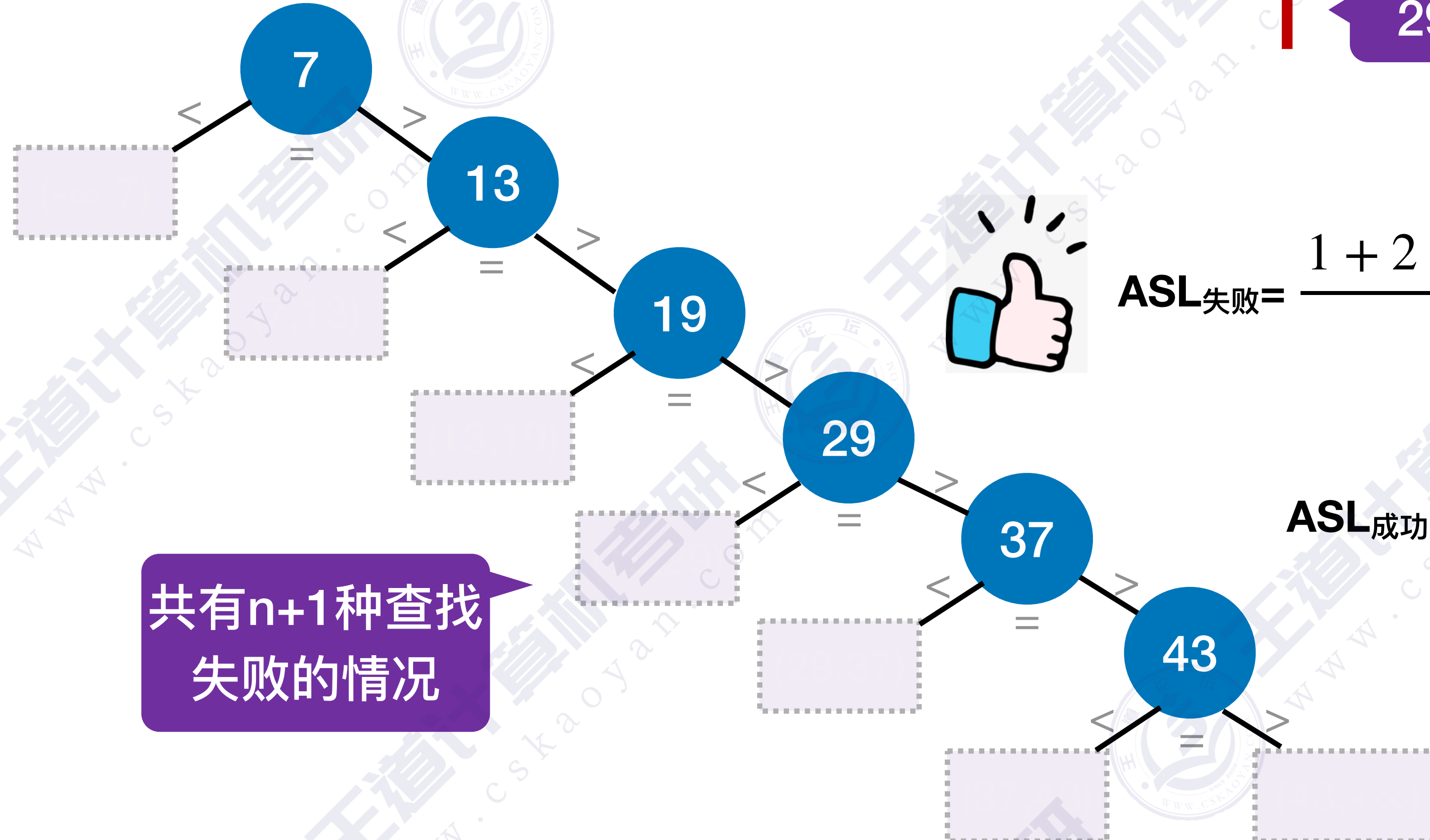
查找表中元素有序存放（递增/递减）

查找目标：

21

7 13 19 29 37 43

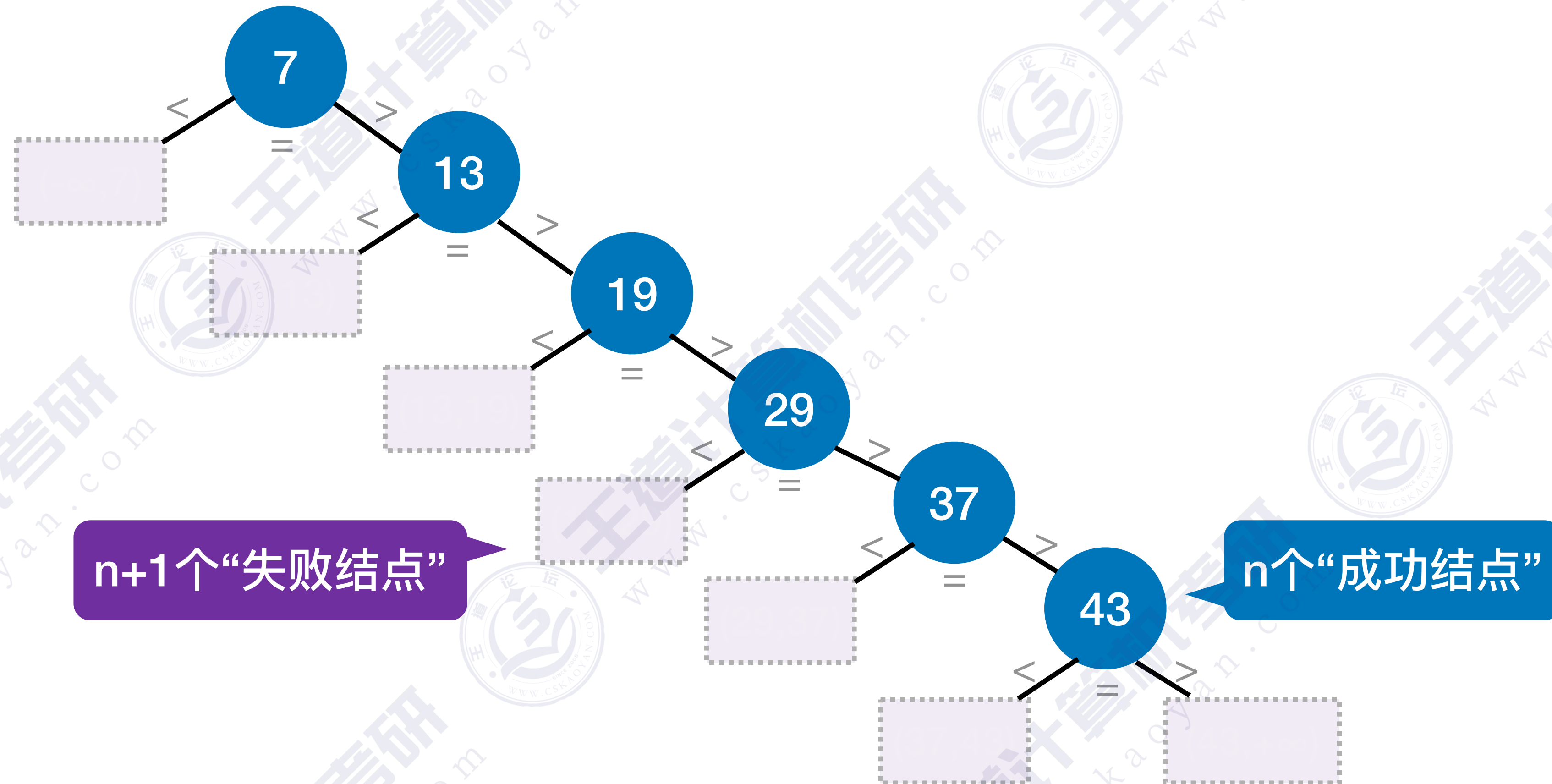
29 > 21 查找失败



$$ASL_{\text{失败}} = \frac{1 + 2 + 3 + \dots + n + n}{n + 1} = \frac{n}{2} + \frac{n}{n + 1}$$

$$ASL_{\text{成功}} = \frac{1 + 2 + 3 + \dots + n}{n} = \frac{n + 1}{2}$$

用查找判定树分析ASL



一个成功结点的查找长度 = 自身所在层数
一个失败结点的查找长度 = 其父节点所在层数
默认情况下, 各种失败情况或成功情况都等概率发生

顺序查找的优化（被查概率不相等）

被查概率

7: 15%

13: 5%

19: 10%

29: 40%

37: 28%

43: 2%

7 13 19 29 37 43



$$ASL_{\text{成功}} = 1 \times 0.15 + 2 \times 0.05 + 3 \times 0.1 + 4 \times 0.4 + 5 \times 0.28 + 6 \times 0.02 = 3.67$$

被查概率大的放在靠前位置

29 37 7 19 13 43



$$ASL_{\text{成功}} = 1 \times 0.4 + 2 \times 0.28 + 3 \times 0.15 + 4 \times 0.1 + 5 \times 0.05 + 6 \times 0.02 = 2.18$$



$$ASL = \sum_{i=1}^n P_i C_i$$

知识回顾与重要考点

顺序查找

算法实现

从头到尾（或者从尾到头）挨个找

适用于顺序表、链表，表中元素有序无序都 OK

可在 0 号位置存“哨兵”，从尾部向头部挨个查找
优点：循环时无需判断下标是否越界

优化

若表中元素有序

当前关键字大于（或小于）目标关键字时，查找失败

优点：查找失败时 ASL 更少

查找判定树

成功结点的关键字对比次数 = 结点所在层数

失败结点的关键字对比次数 = 其父节点所在层数

若各个关键字被查概率不同

可按被查概率降序排列

优点：查找成功时 ASL 更少

时间复杂度

$O(n)$

本节内容

折半查找

知识总览

折半查找

算法思想

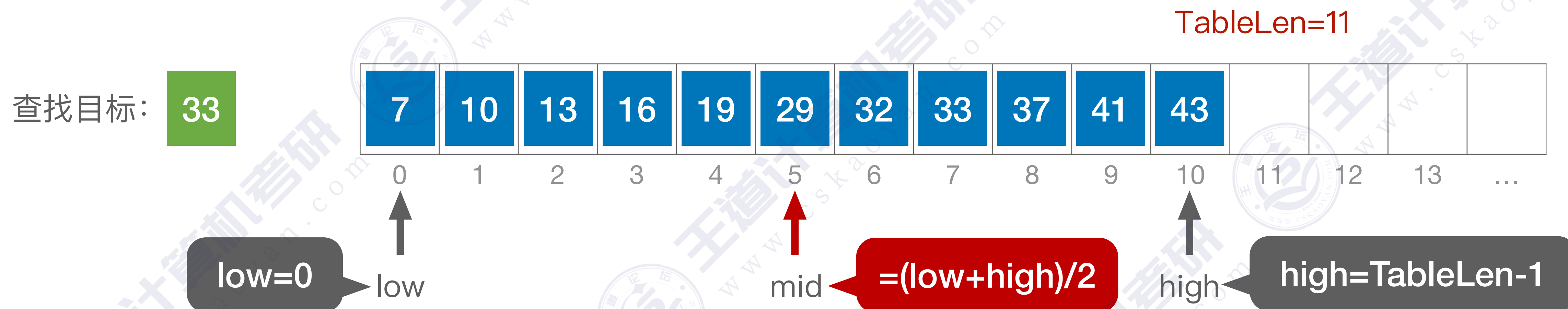
算法实现

查找判定树

折半查找效率

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



33>mid, 只可能在右边区域

将头发梳成大人模样

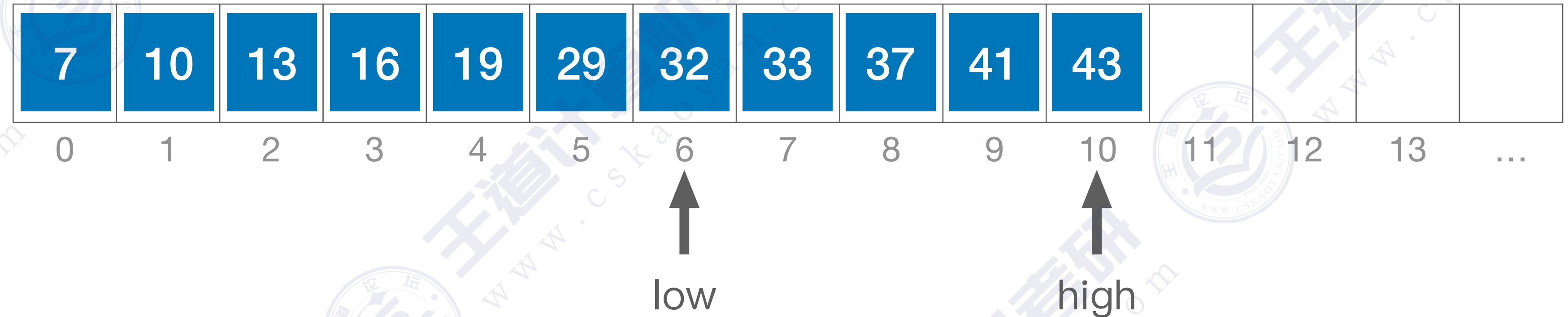
折半查找的算法思想



折半查找，又称“二分查找”，仅适用于**有序的顺序表**。

查找目标：

33



注：只有在 $[low, high]$ 之间才有可能找到目标关键字

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。

查找目标：

33

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
						low		mid		high				

33<mid, 只可能在左边区域

折半查找的算法思想



折半查找，又称“二分查找”，仅适用于**有序的顺序表**。

查找目标：

33

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
						↑	↑							
						low	high							

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。

查找目标：

33

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

low

high

mid

33>mid, 只可能在右边区域

折半查找的算法思想



折半查找，又称“二分查找”，仅适用于**有序的顺序表**。

查找目标：

33

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

↑
high

↑
low

折半查找的算法思想



折半查找，又称“二分查找”，仅适用于**有序的顺序表**。

查找目标：

33

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

high

low

mid

33==mid
查找成功

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



折半查找的算法思想



折半查找，又称“二分查找”，仅适用于**有序**的**顺序表**。

查找目标：

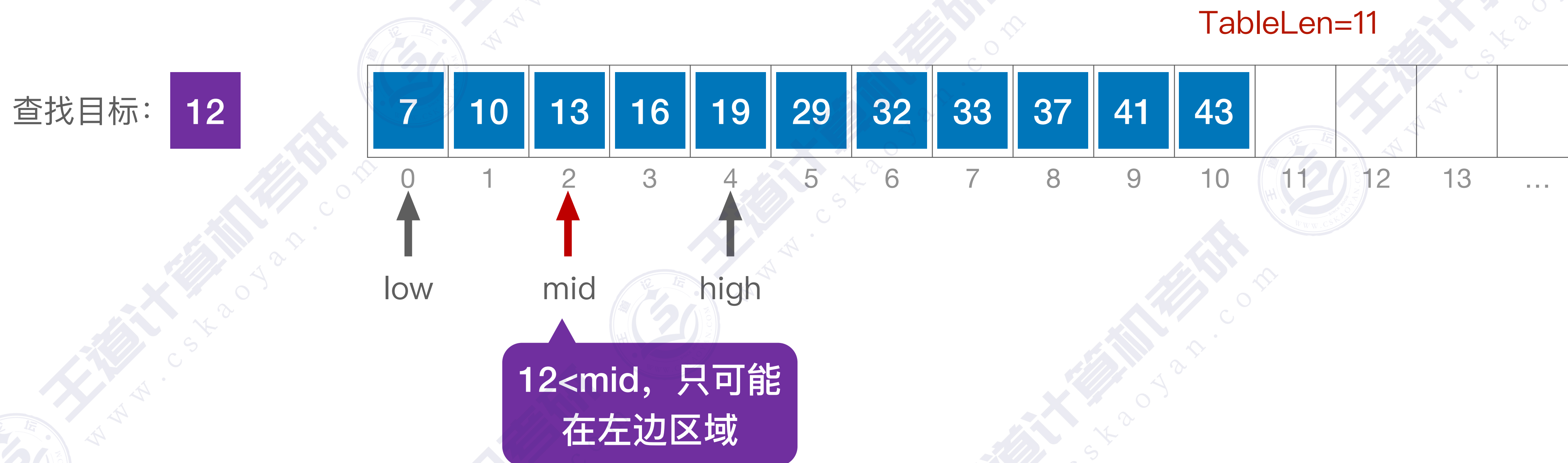
12

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
↑				↑										
low				high										

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



折半查找的算法思想



折半查找，又称“二分查找”，仅适用于**有序**的**顺序表**。

查找目标：

12

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
↑	↑													
low	high													

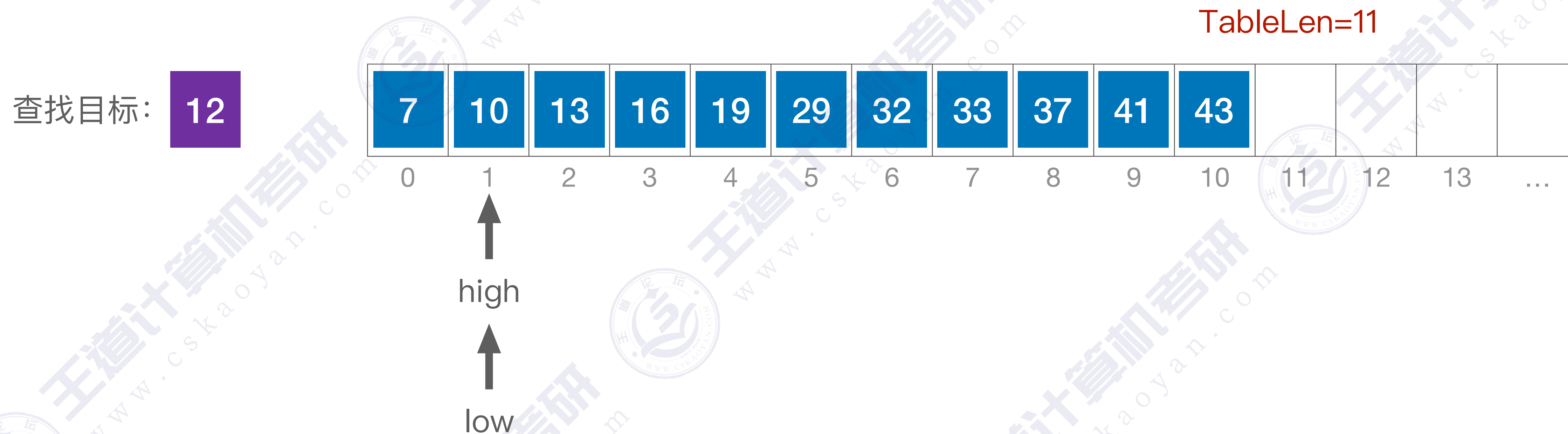
折半查找的算法思想

折半查找，又称“二分查找”，仅适用于**有序的顺序表**。



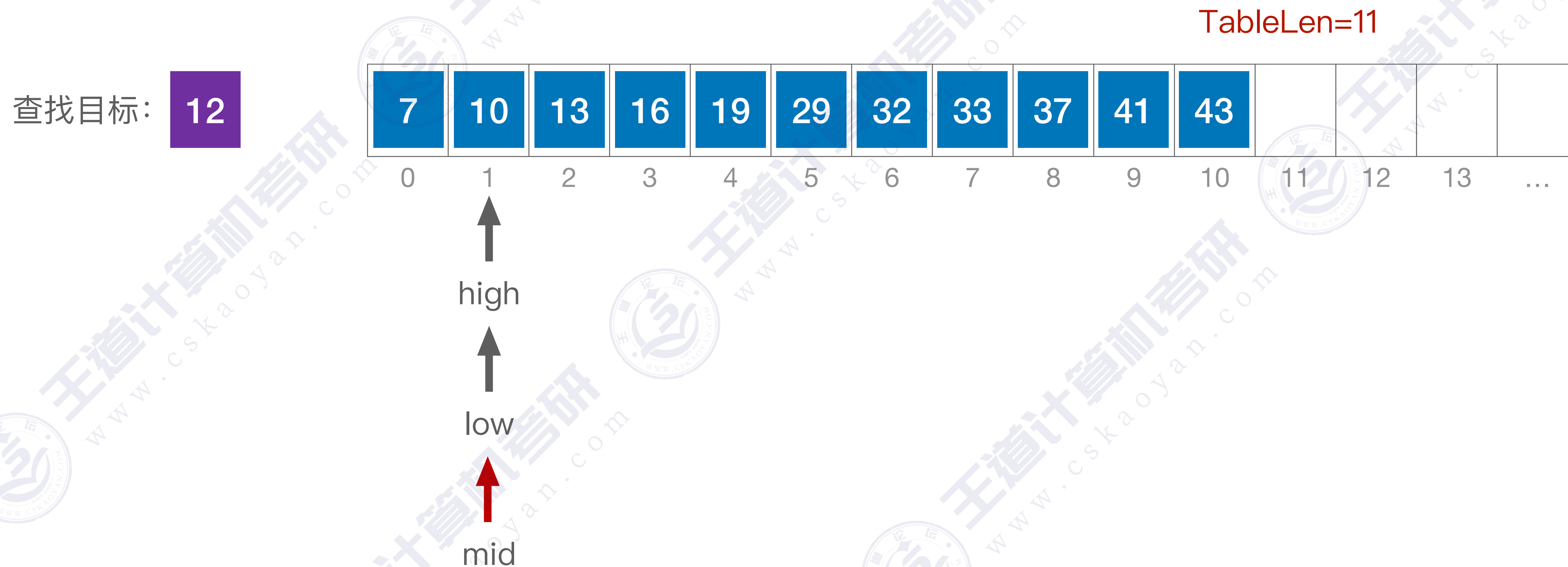
折半查找的算法思想

折半查找，又称“二分查找”，仅适用于**有序的顺序表**。



折半查找的算法思想

折半查找，又称“二分查找”，仅适用于**有序的顺序表**。



折半查找的算法思想



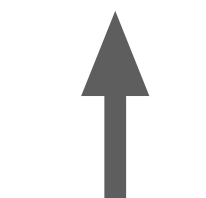
折半查找，又称“二分查找”，仅适用于**有序**的**顺序表**。

查找目标：

12

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...



high



low

low>high
查找失败

折半查找的实现

```
typedef struct{
    ElemType *elem;
    int TableLen;
}SSTable;
```

//查找表的数据结构 (顺序表)
//动态数组基址
//表的长度

折半查找，又称“二分查找”，
仅适用于有序的顺序表。

顺序表拥有随机访问的特性，链表没有

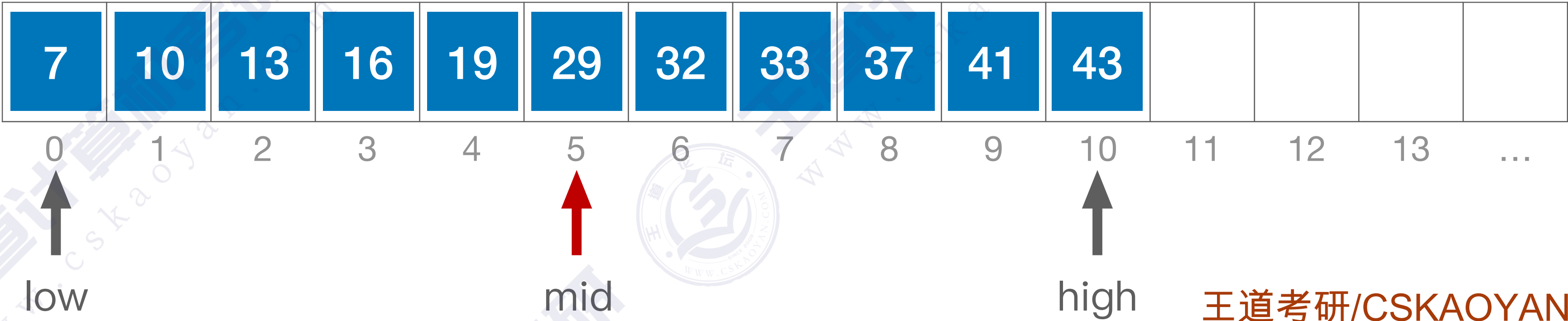
```
//折半查找
int Binary_Search(SSTable L,ElemType key){
    int low=0,high=L.TableLen-1,mid;
    while(low<=high){
        mid=(low+high)/2;
        if(L.elem[mid]==key)
            return mid;
        else if(L.elem[mid]>key)
            high=mid-1;
        else
            low=mid+1;
    }
    return -1;
}
```

//取中间位置
//查找成功则返回所在位置
//从前半部分继续查找
//从后半部分继续查找
//查找失败，返回-1

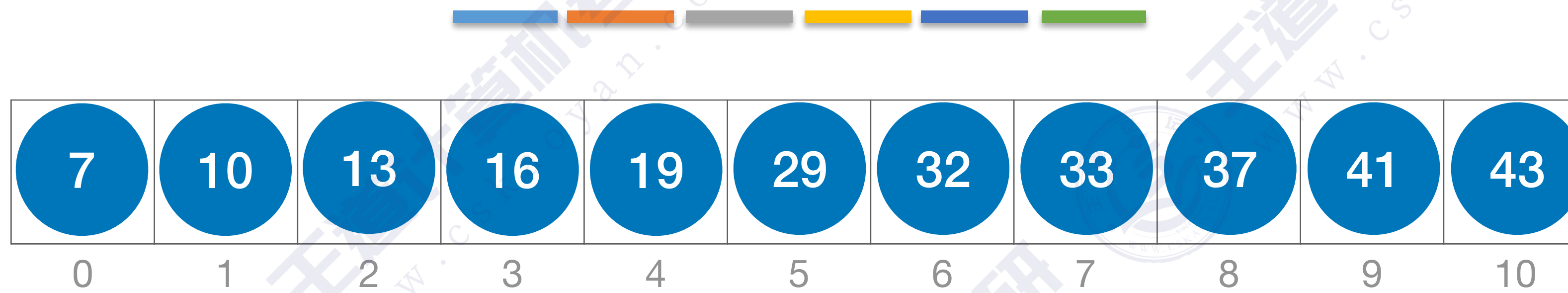
TableLen=11

查找目标:

33



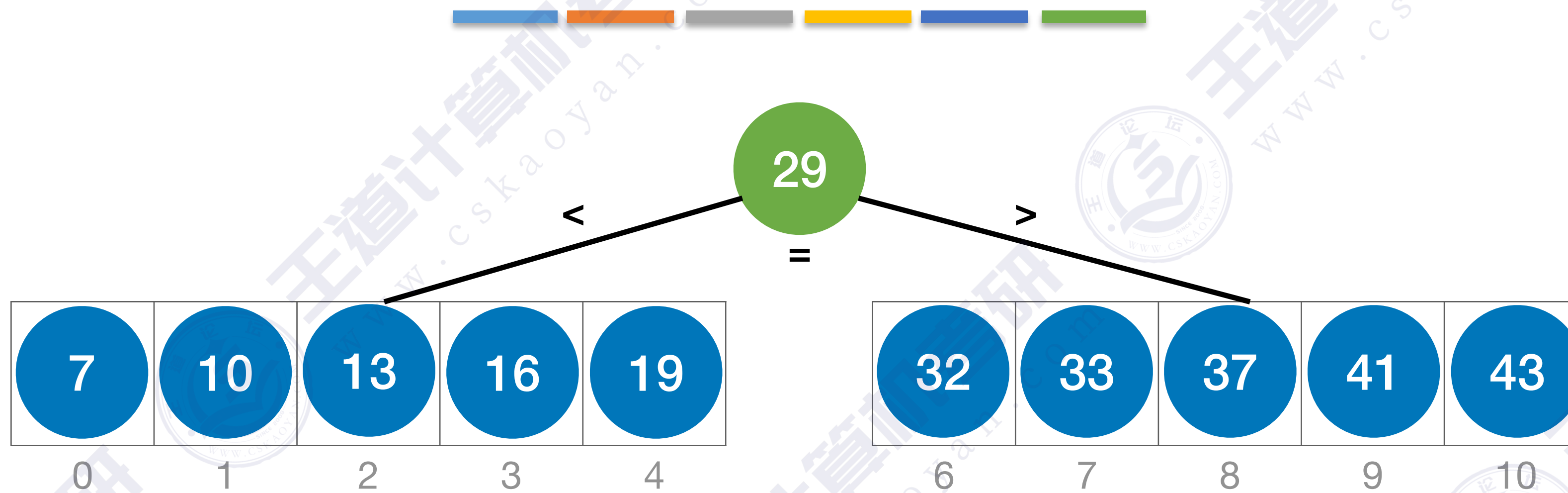
查找效率分析



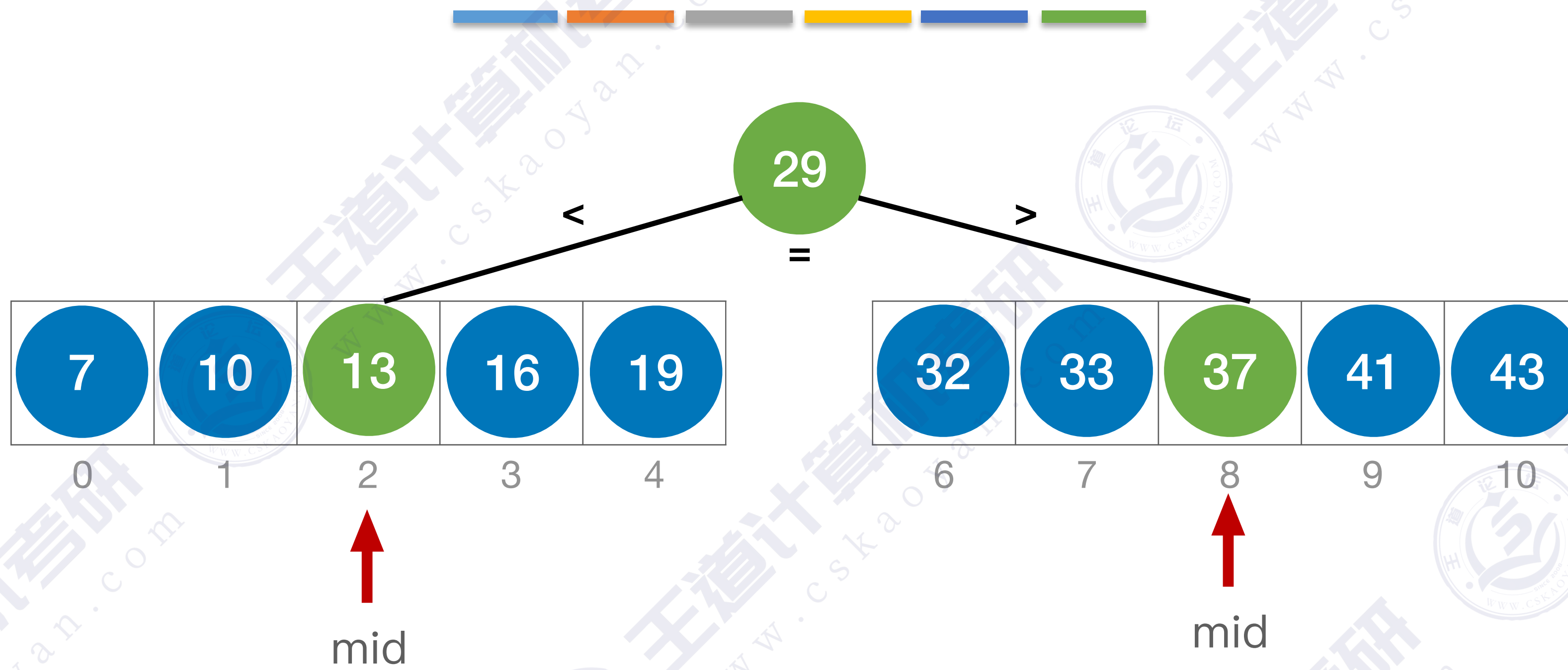
查找效率分析



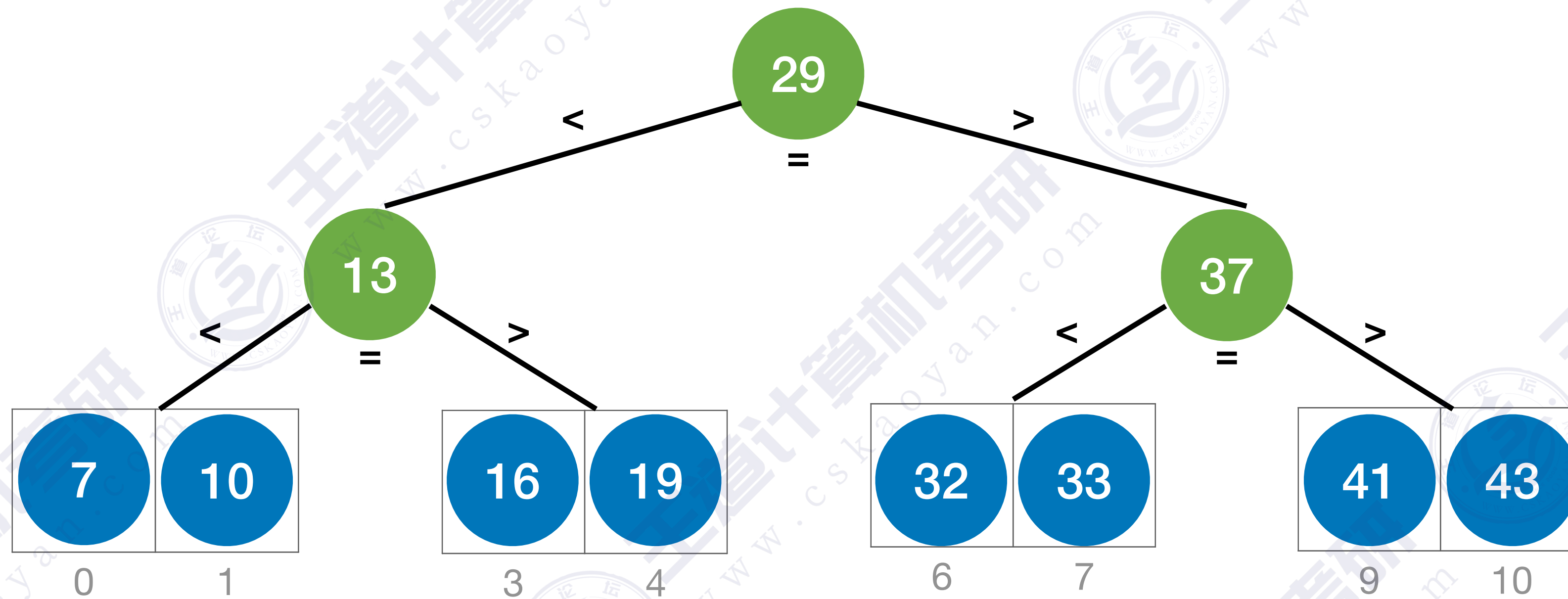
查找效率分析



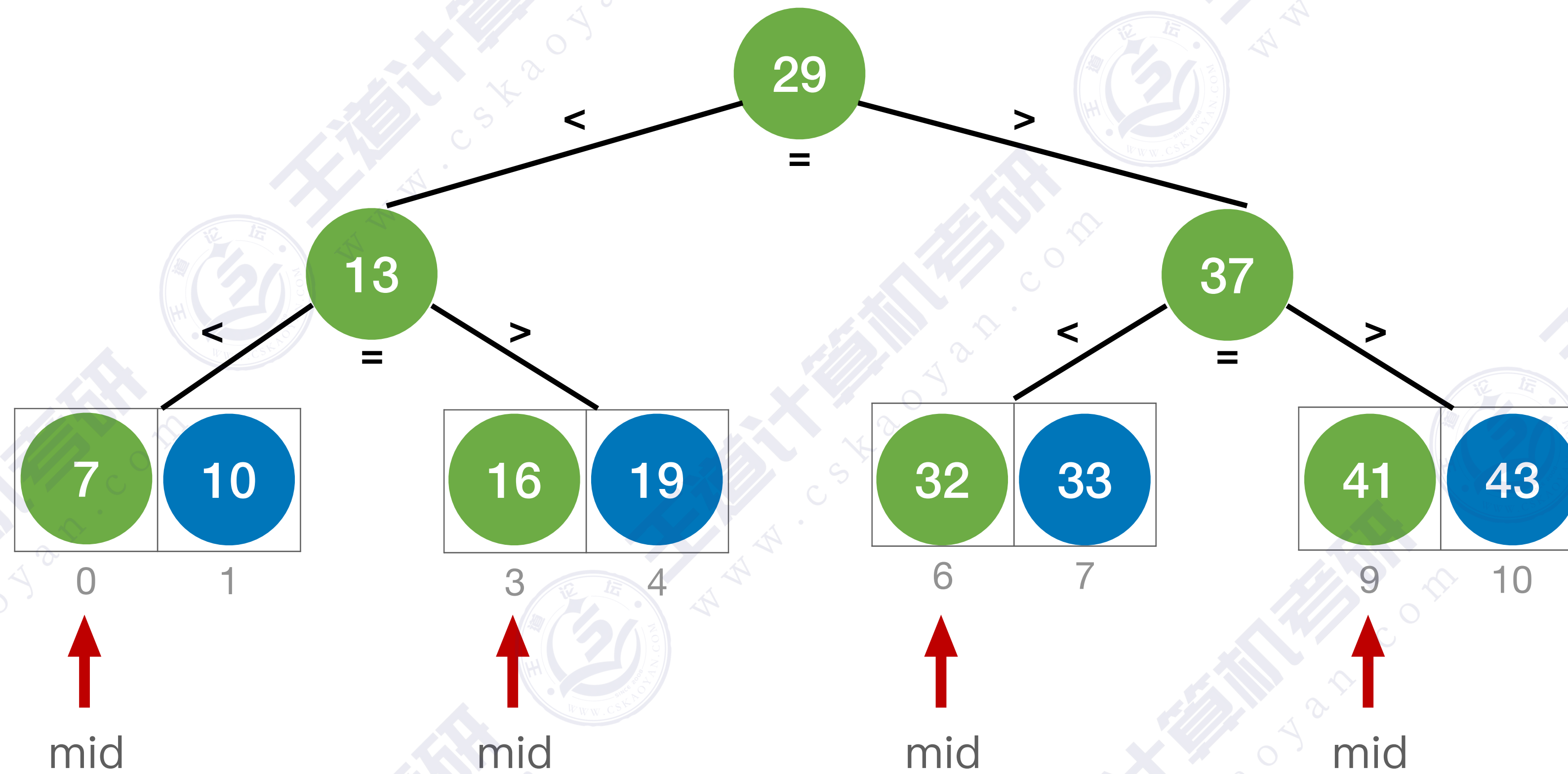
查找效率分析



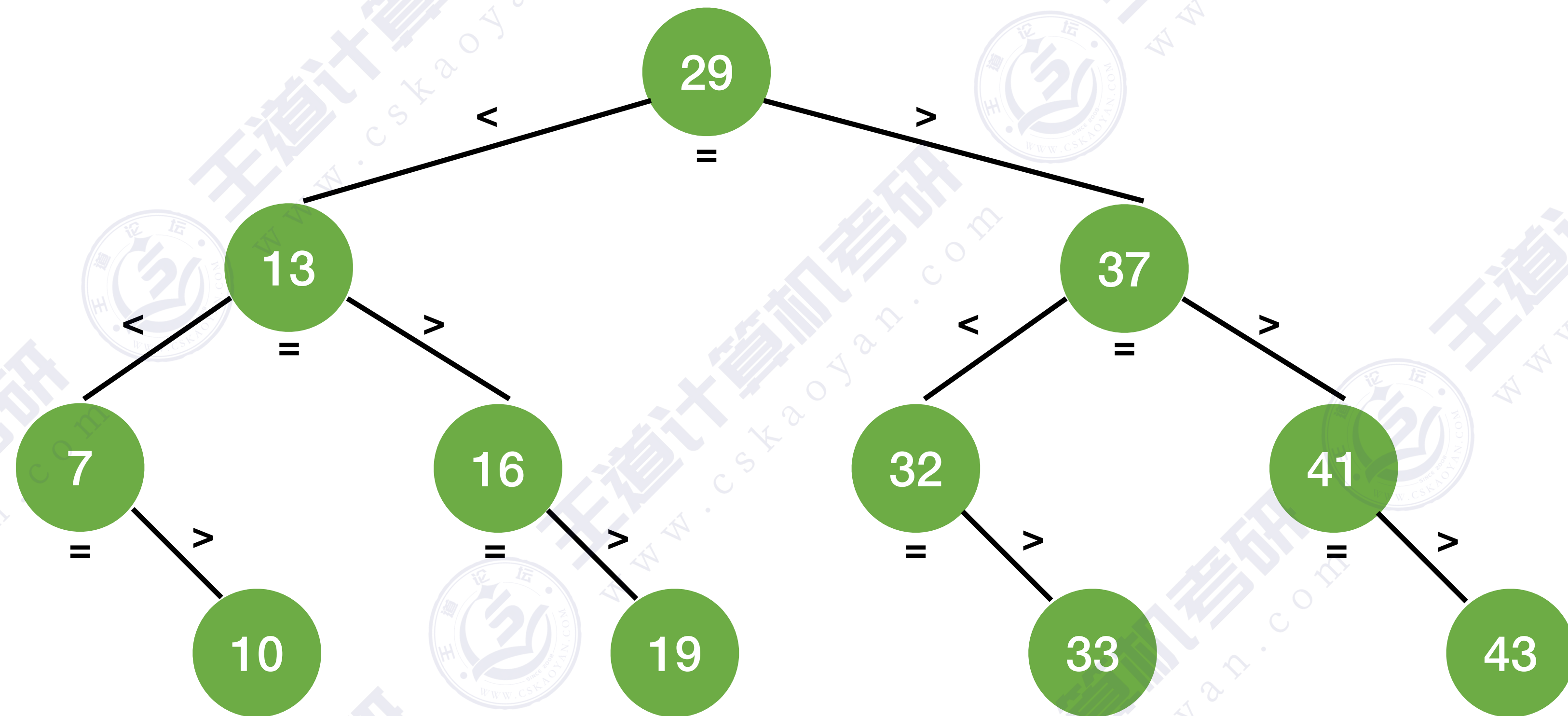
查找效率分析



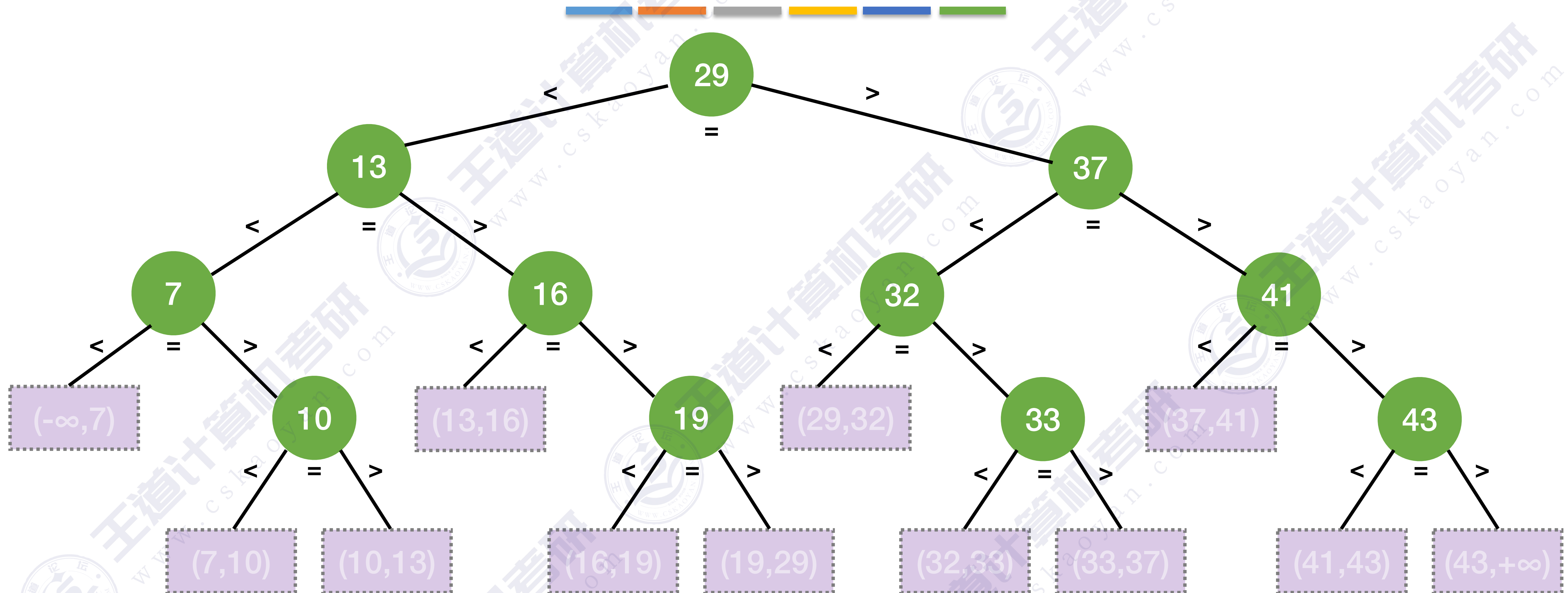
查找效率分析



查找效率分析



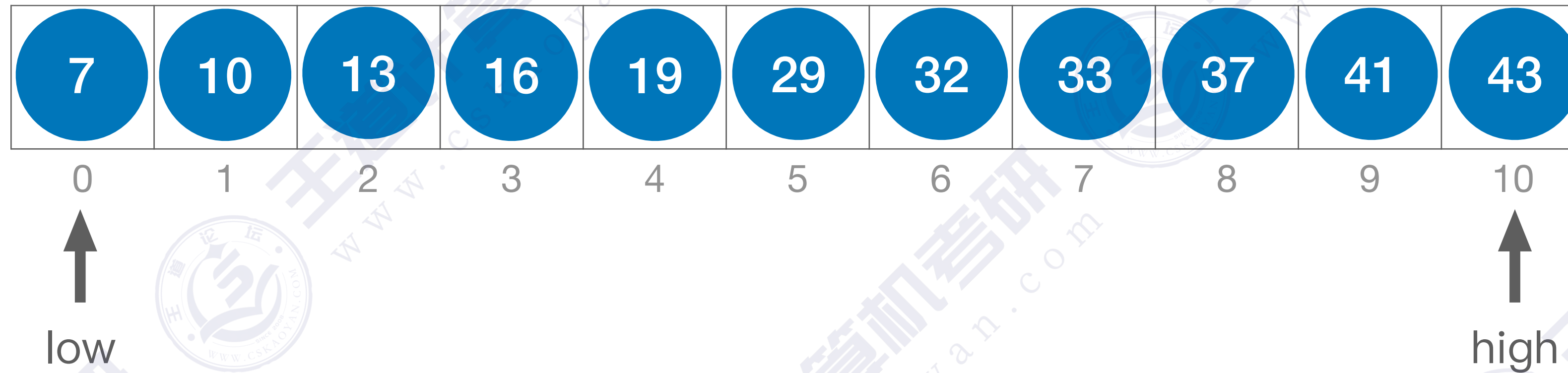
查找效率分析



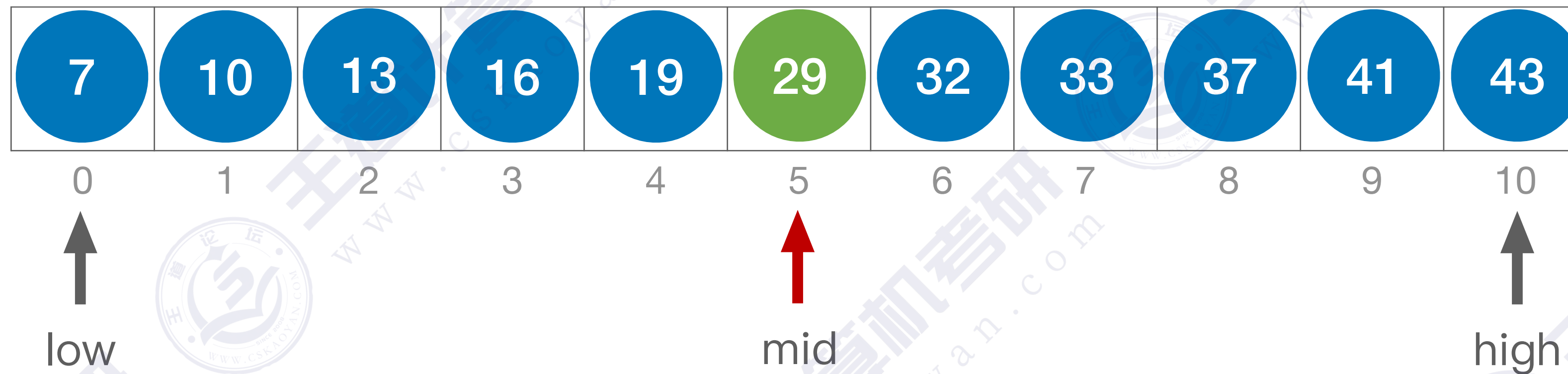
$$ASL_{\text{成功}} = (1 \cdot 1 + 2 \cdot 2 + 3 \cdot 4 + 4 \cdot 4) / 11 = 3$$

$$ASL_{\text{失败}} = (3 \cdot 4 + 4 \cdot 8) / 12 = 11/3$$

折半查找判定树的构造



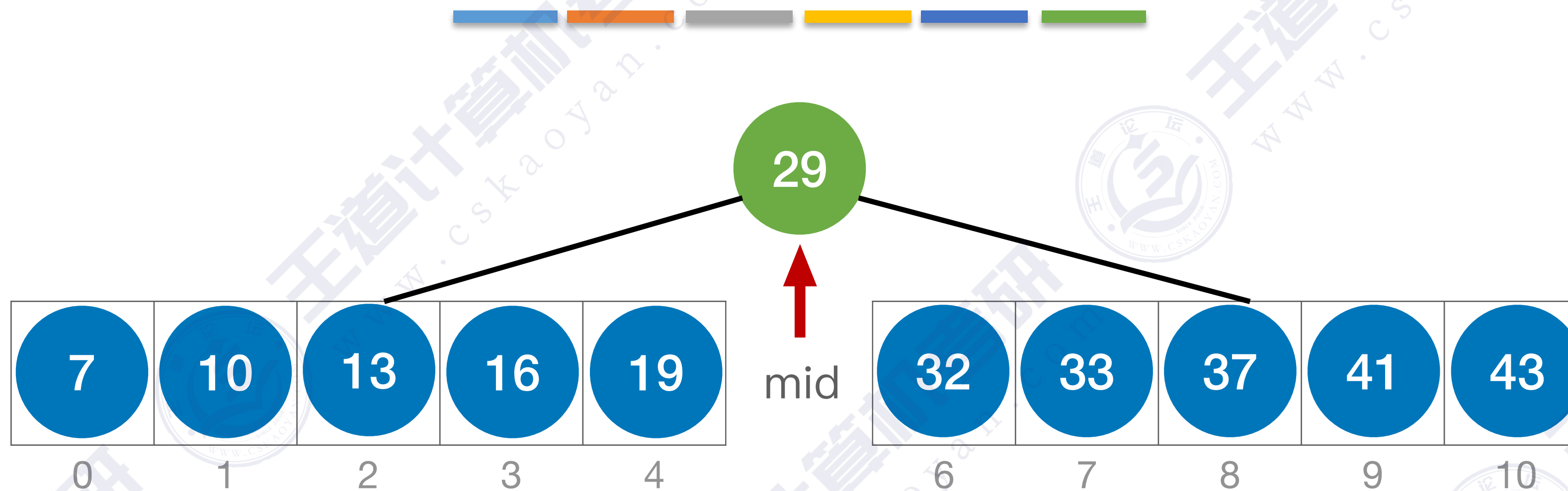
折半查找判定树的构造



$$mid = \lfloor (low + high) / 2 \rfloor$$

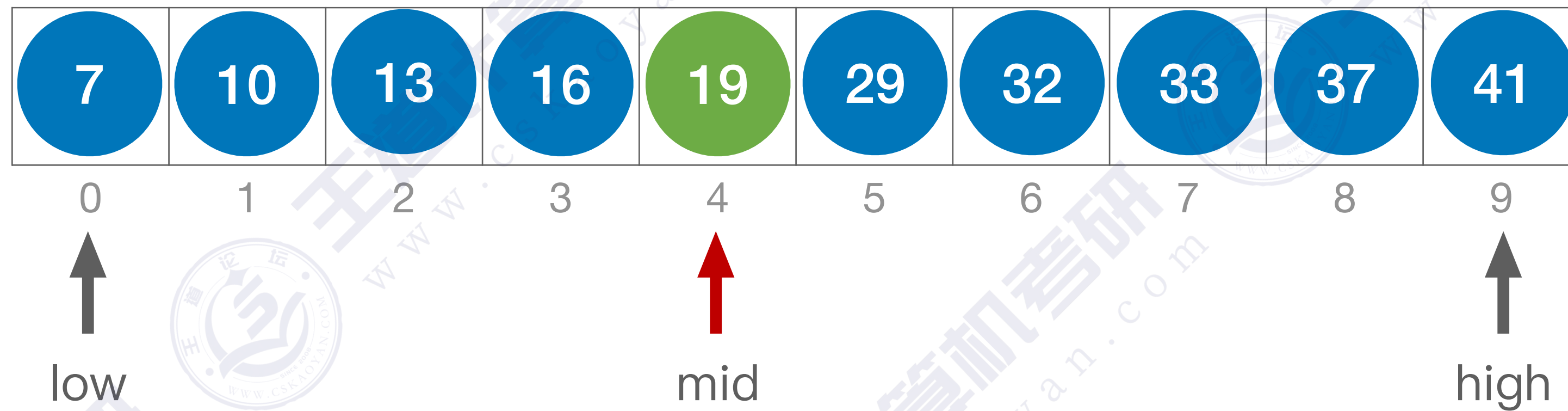
如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

折半查找判定树的构造



如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

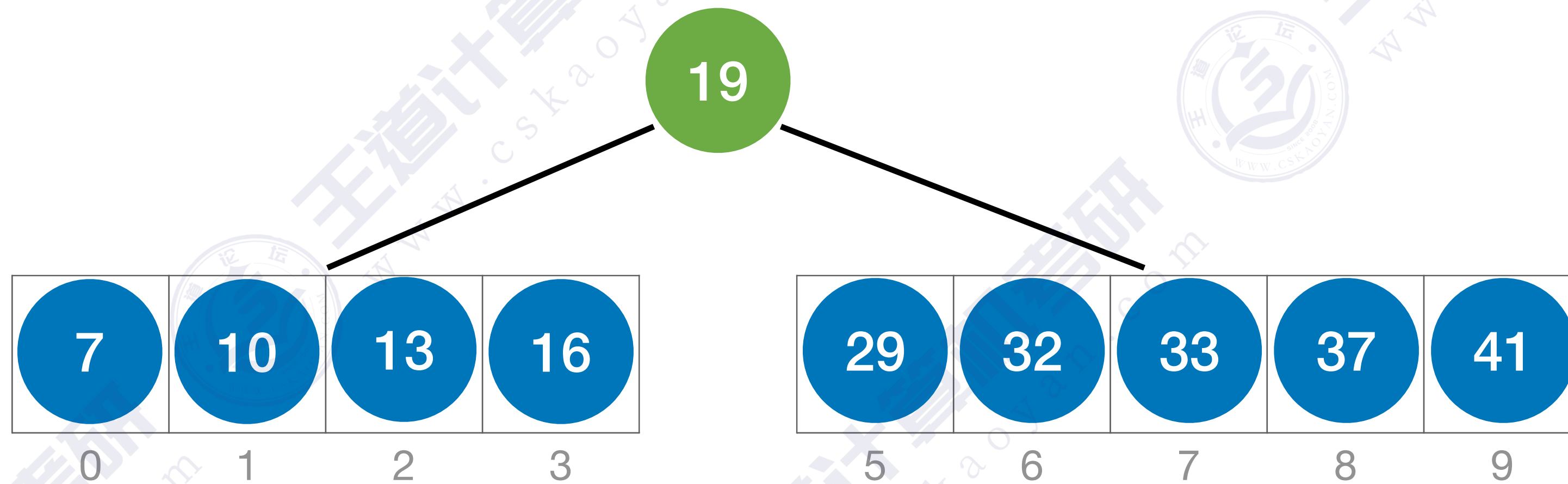
折半查找判定树的构造



$$mid = \lfloor (low + high) / 2 \rfloor$$

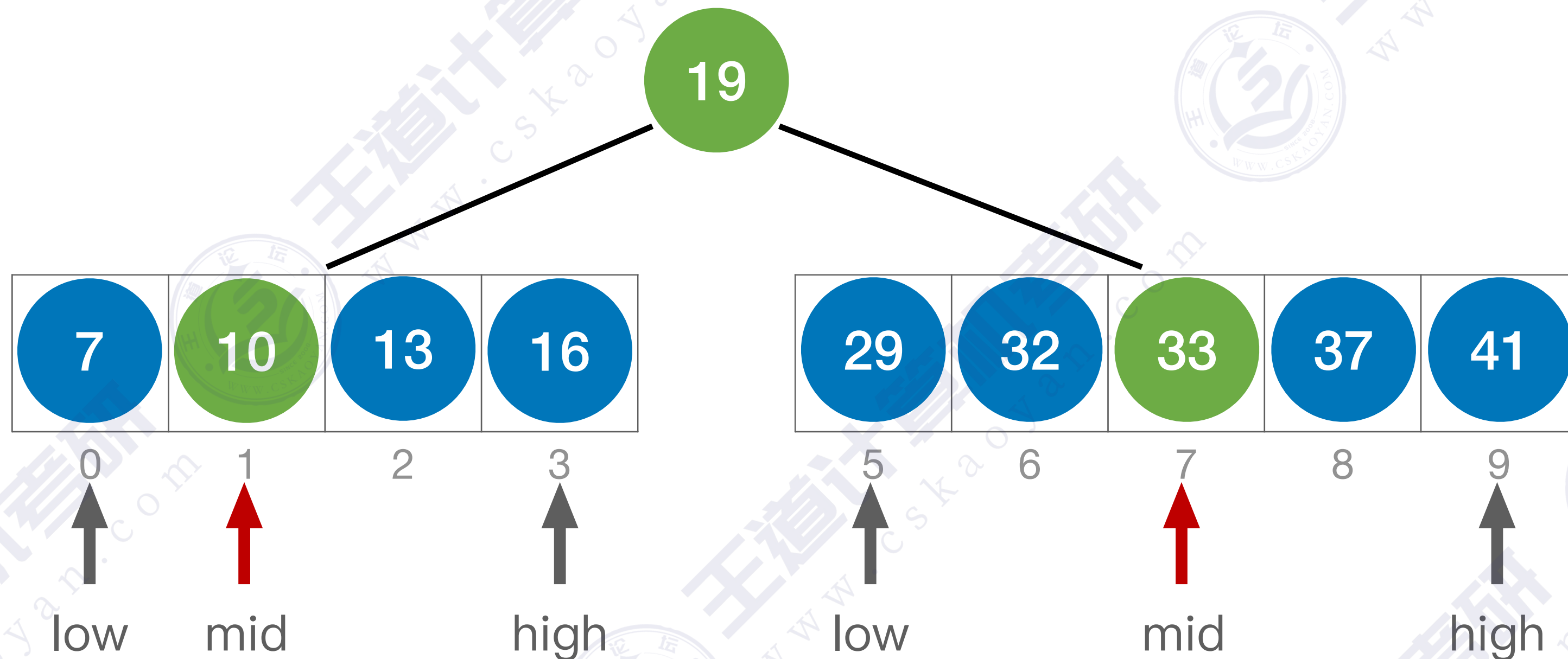
如果当前 low 和 $high$ 之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造



如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造

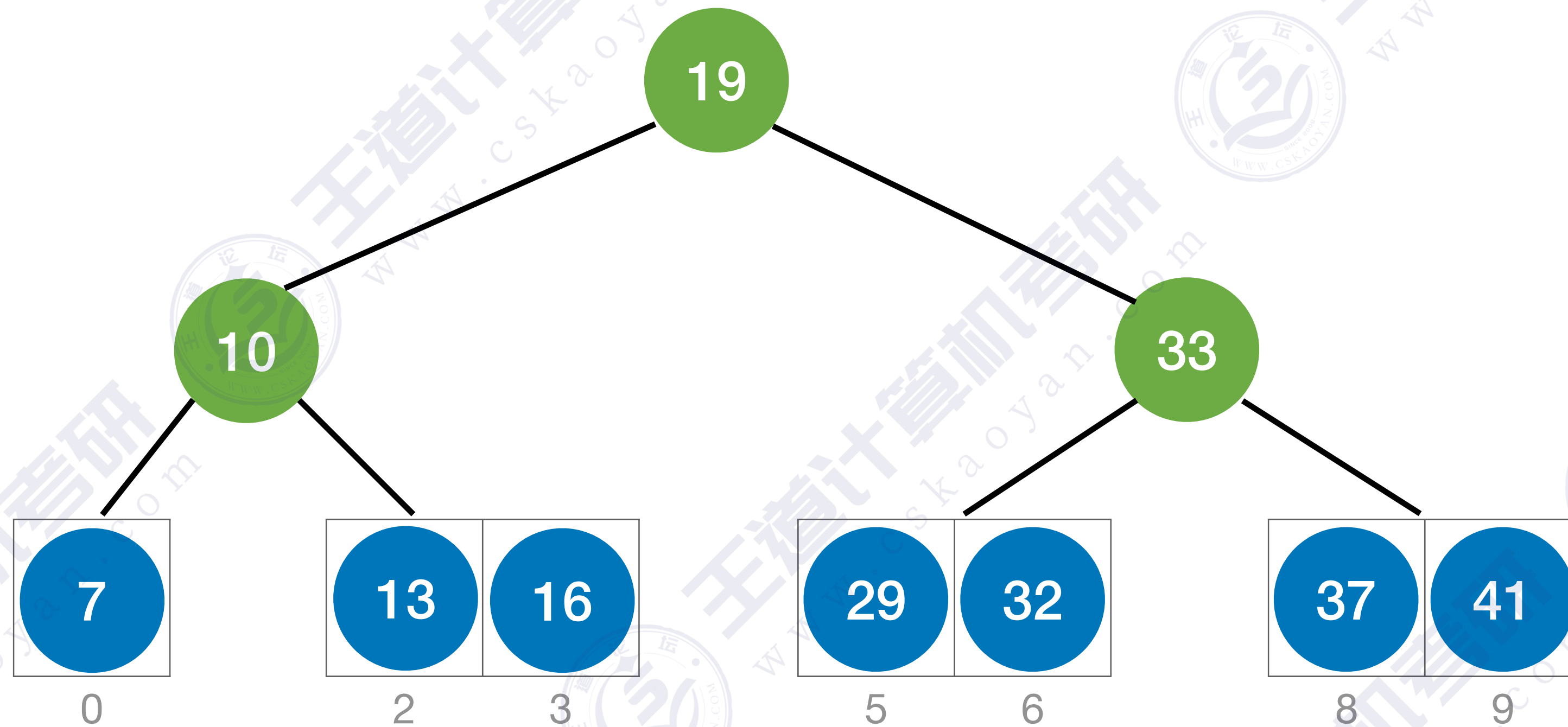


$$mid = \lfloor (low + high) / 2 \rfloor$$

如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

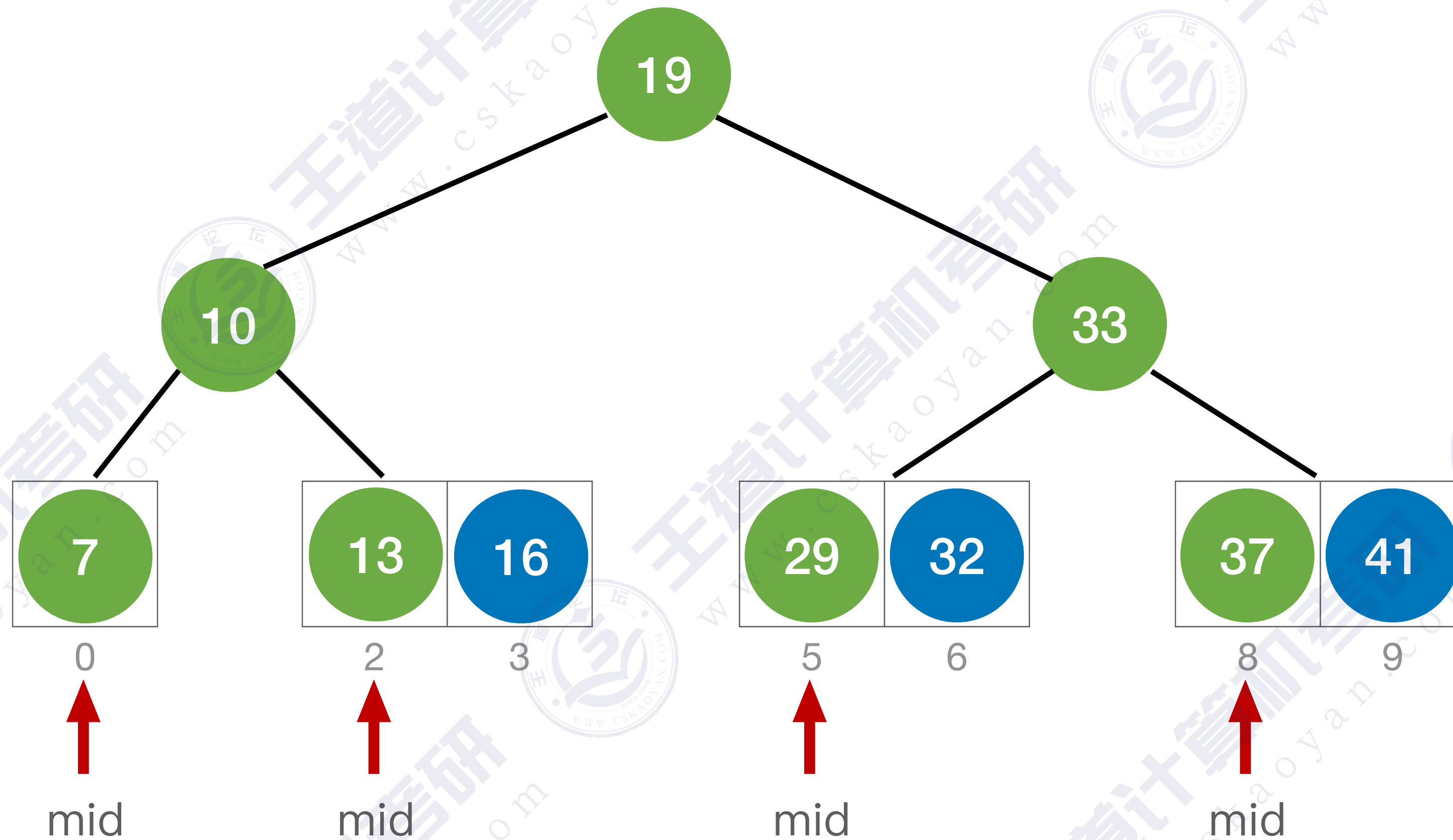
折半查找判定树的构造



如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

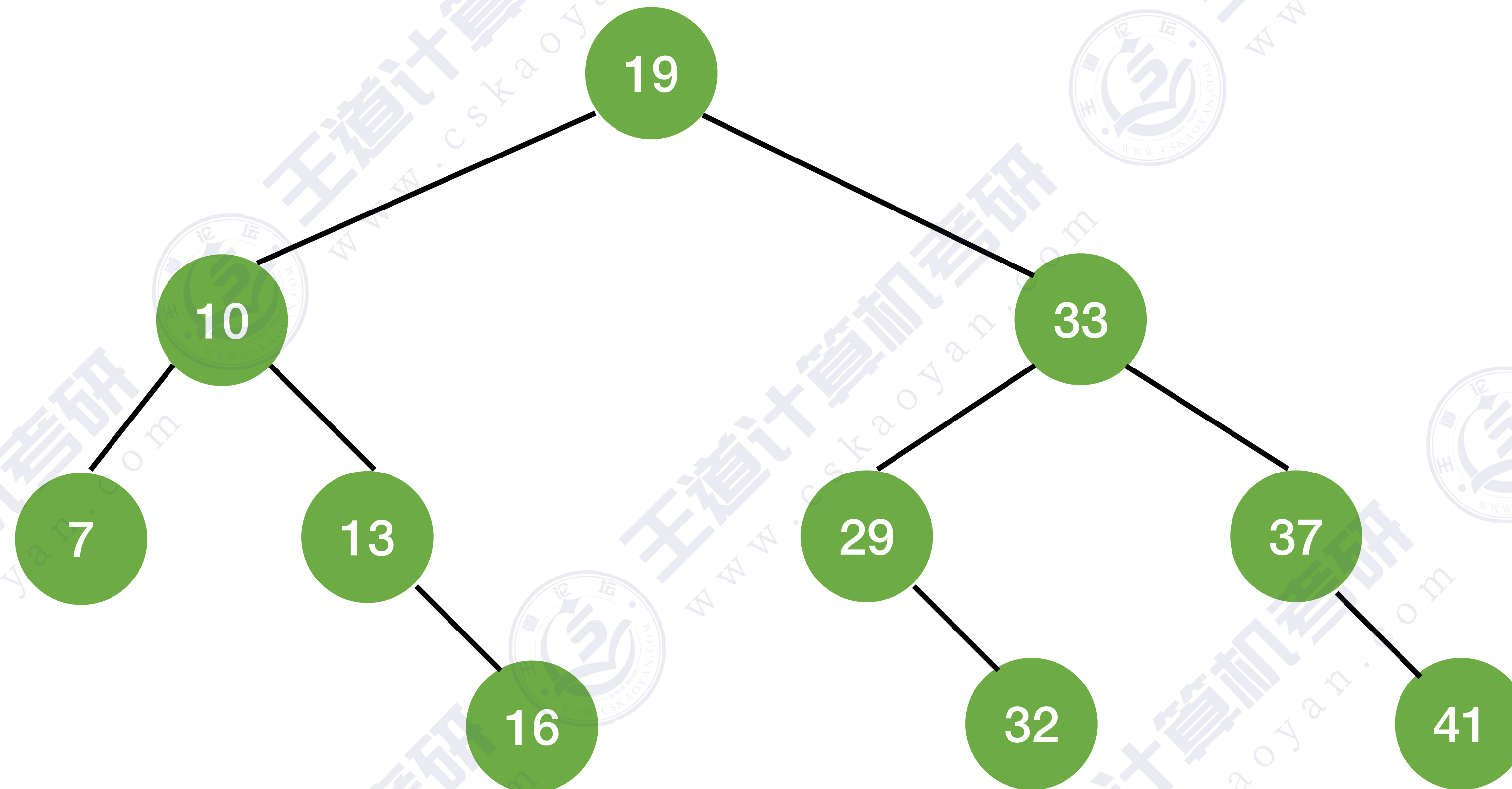
折半查找判定树的构造



如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造



如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造



如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

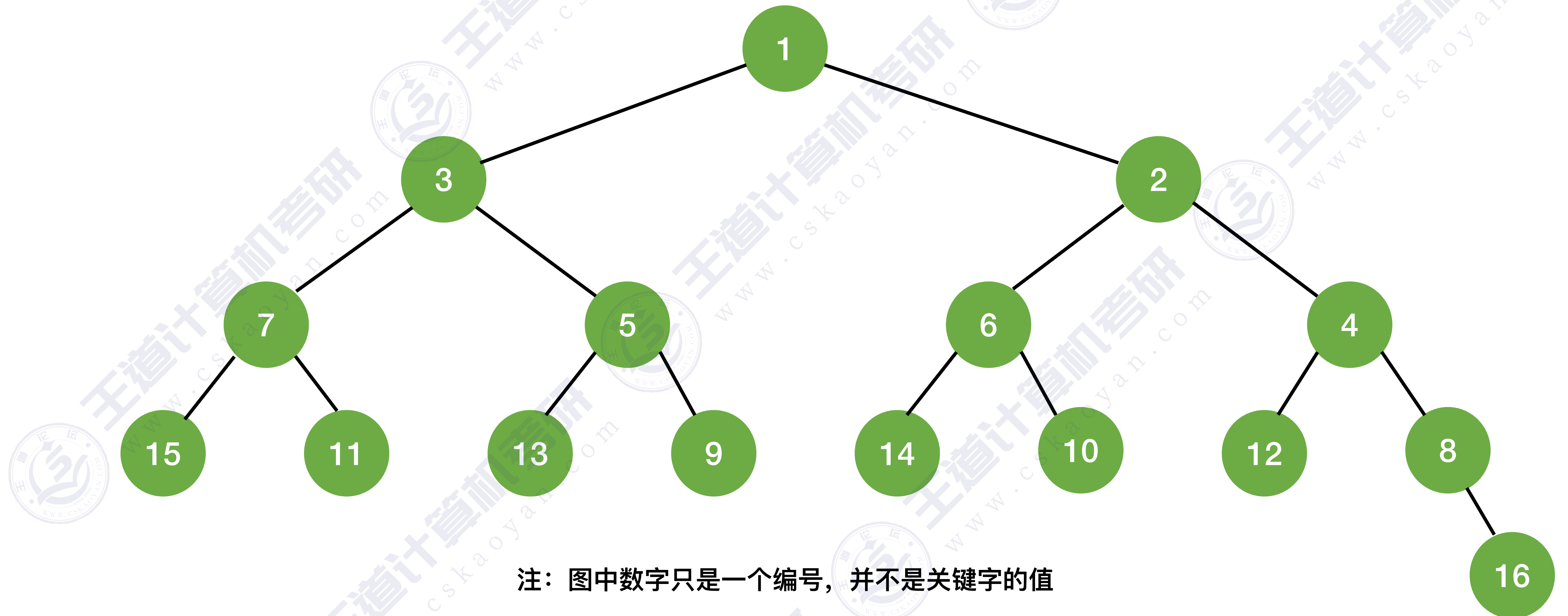


折半查找的判定树中，若 $mid = \lfloor (low + high)/2 \rfloor$ ，则对于任何一个结点，必有：

右子树结点数-左子树结点数=0或1

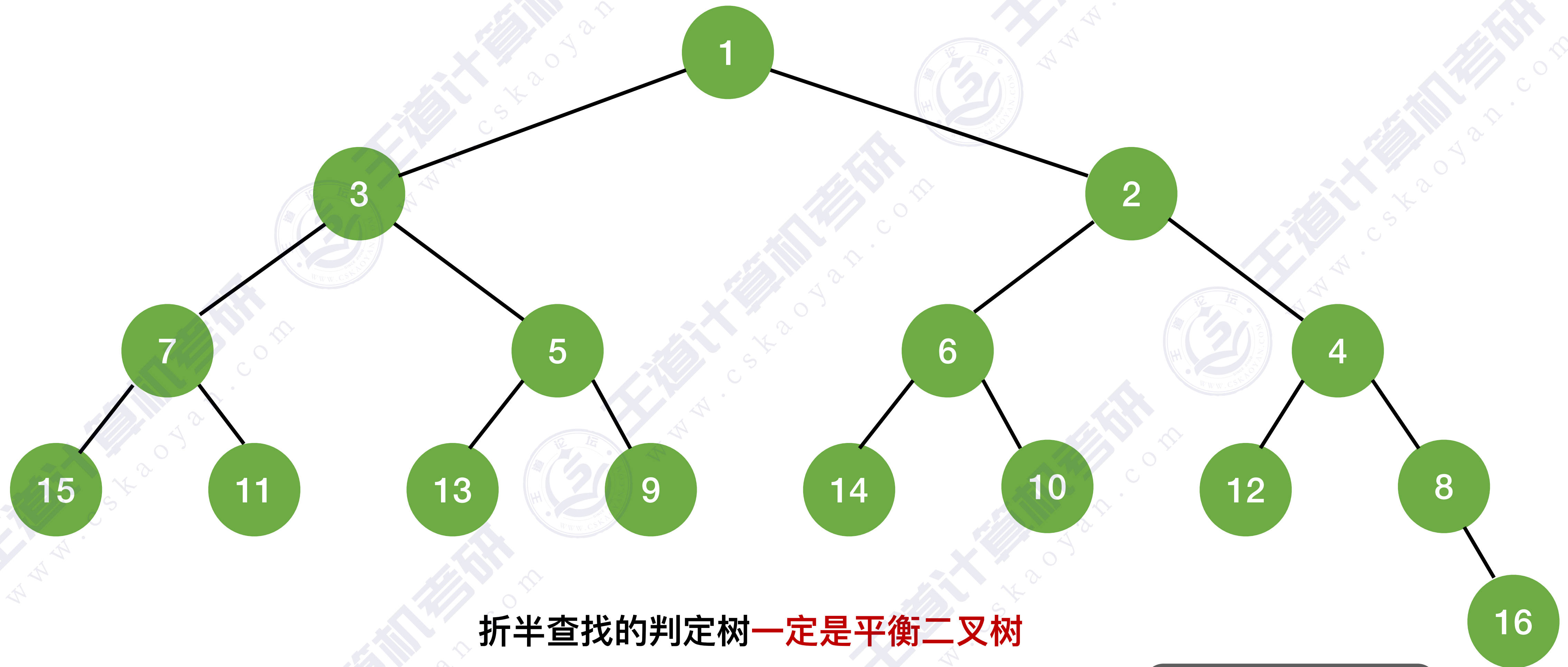
折半查找判定树的构造

练习：若 $mid = \lfloor (low + high)/2 \rfloor$ ，画出含1个元素、2个元素、3个元素...16个元素 的查找表对应的折半查找判定树，注：暂不考虑失败结点（Key：右子树结点数-左子树结点数=0或1）



注：图中数字只是一个编号，并不是关键字的值

折半查找判定树的构造

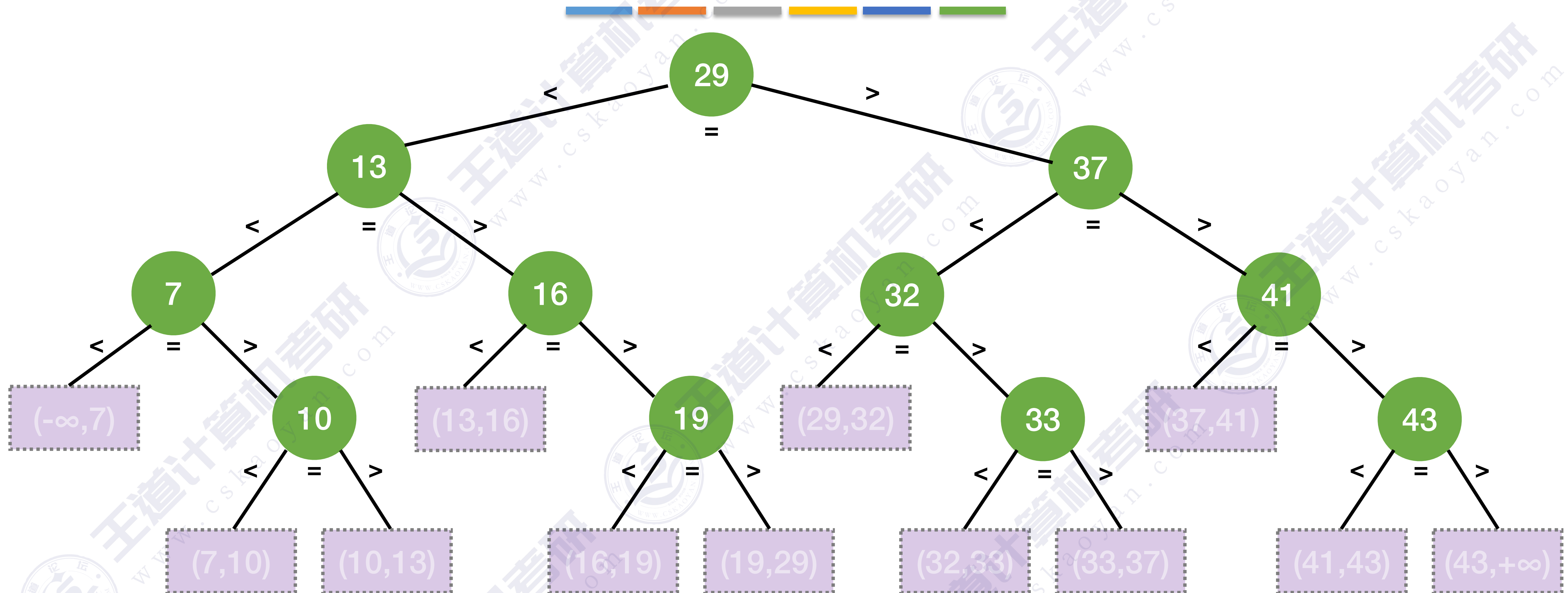


折半查找的判定树**一定是平衡二叉树**

折半查找的判定树中，**只有最下面一层是不满的**
因此，元素个数为 n 时**树高** $h = \lceil \log_2(n + 1) \rceil$

注：计算方法同
“完全二叉树”

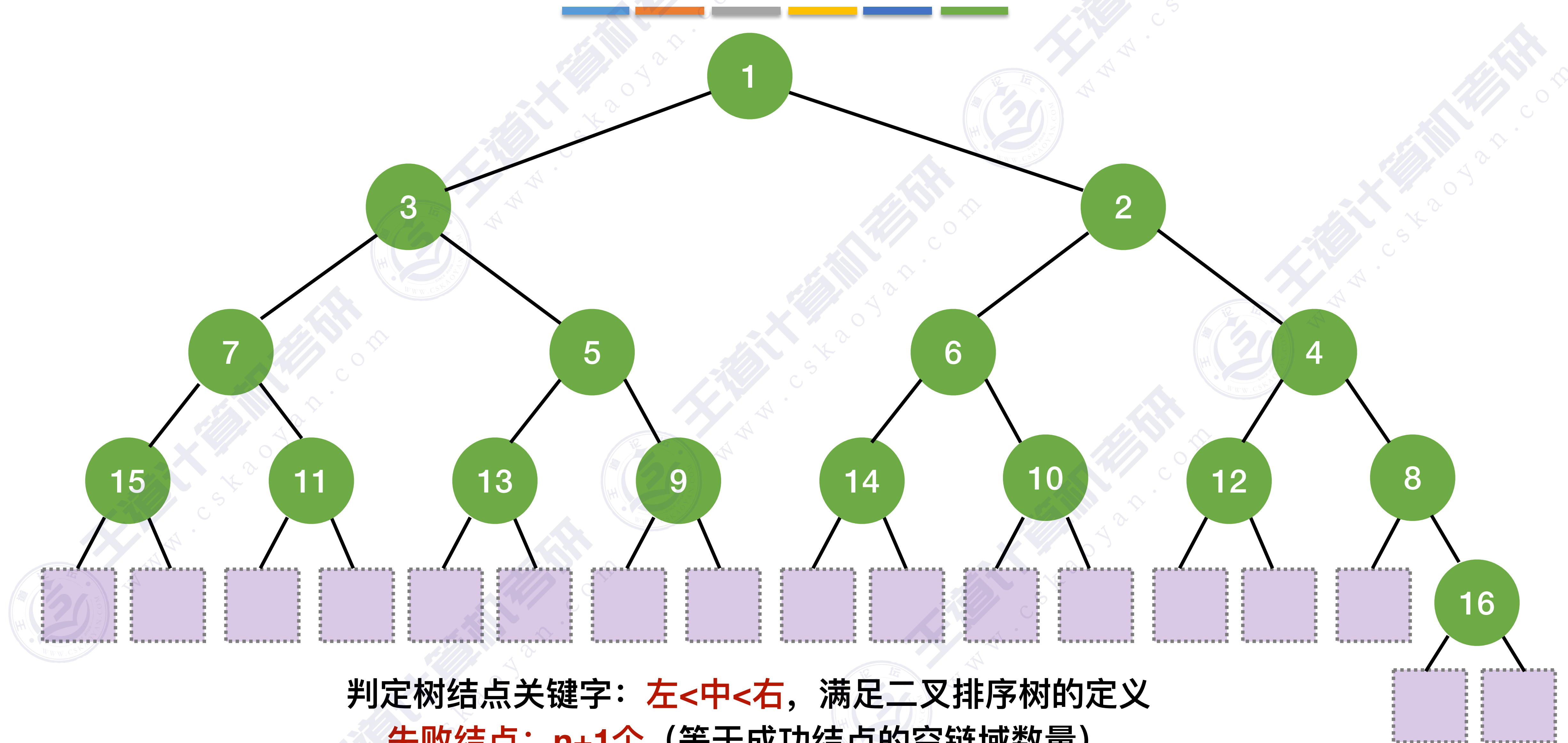
折半查找判定树的构造



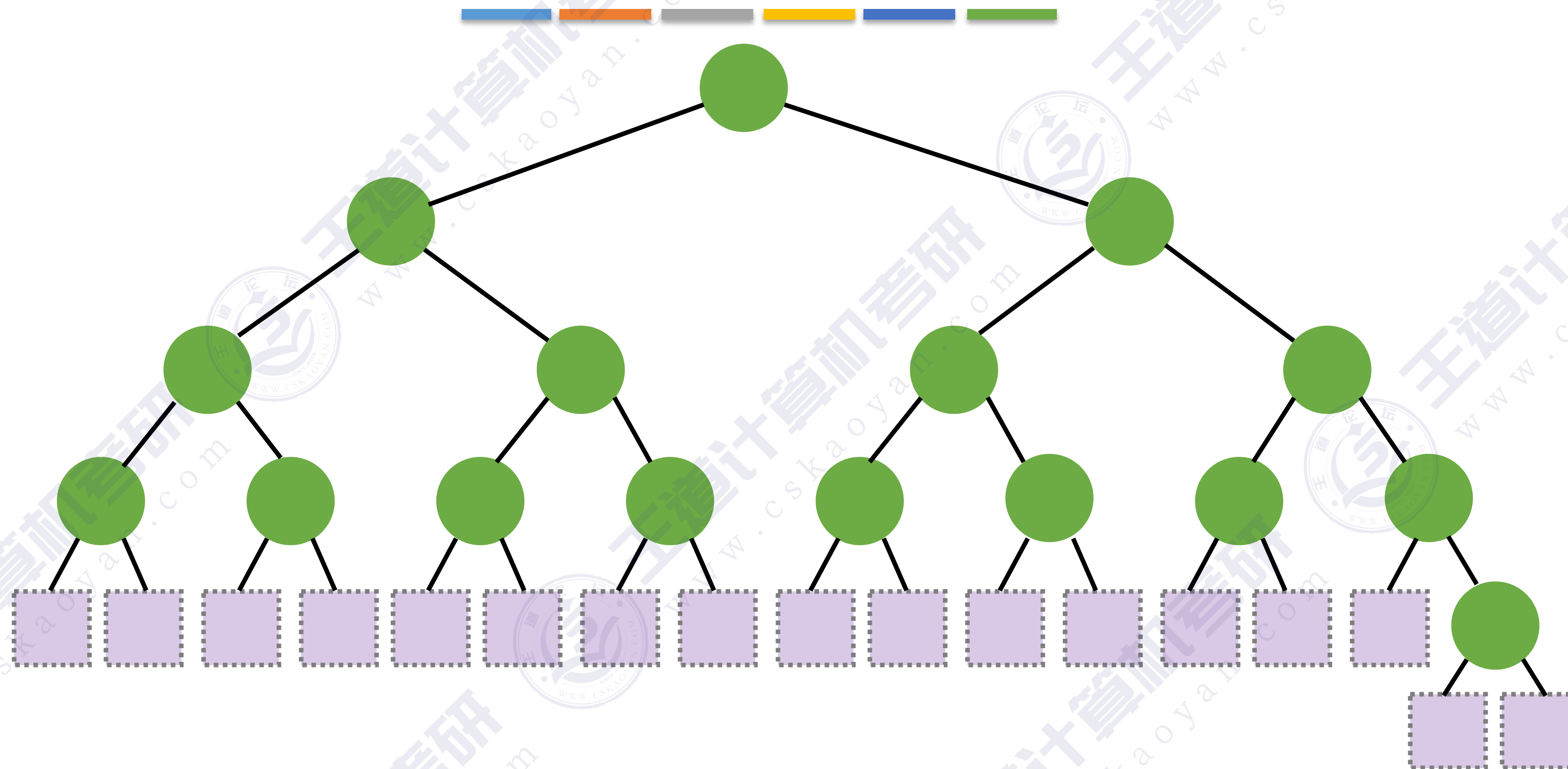
判定树结点关键字：左<中<右，满足二叉排序树的定义

失败结点： $n+1$ 个（等于成功结点的空链域数量）

折半查找判定树的构造



折半查找的查找效率



树高 $h = \lceil \log_2(n + 1) \rceil$

查找成功的ASL $\leq h$

折半查找的时间复杂度 = $O(\log_2 n)$

注：该树高不包含失败结点

查找失败的ASL $\leq h$

知识回顾与重要考点

折半查找

适用范围



只适用于有序的顺序表

算法思想



在 $[low, high]$ 之间找目标关键字，每次检查 $mid = (low + high) / 2$

根据 mid 所指元素与目标关键字的大小调整 low 或 $high$ ，不断缩小 low 和 $high$ 的范围

若 $low > high$ 则查找失败

判定树



构造



由 mid 所指元素将原有元素分割到左右子树中

Key: 右子树结点数 - 左子树结点数 = 0 或 1

折半查找的判定树是平衡的二叉排序树 (左 < 中 < 右)

折半查找判定树，只有最下面一层是不满的

特性



若查找表有 n 个关键字，则失败结点有 $n + 1$ 个

树高 $h = \lceil \log_2(n + 1) \rceil$

(不包含失败结点)

时间复杂度



$O(\log_2 n)$

拓展思考



折半查找时间复杂度= $O(\log_2 n)$
顺序查找的时间复杂度= $O(n)$

辣么，折半查找的速度一定比顺序查找更快？

查找目标：

7

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

否认三连



我不是



我没有

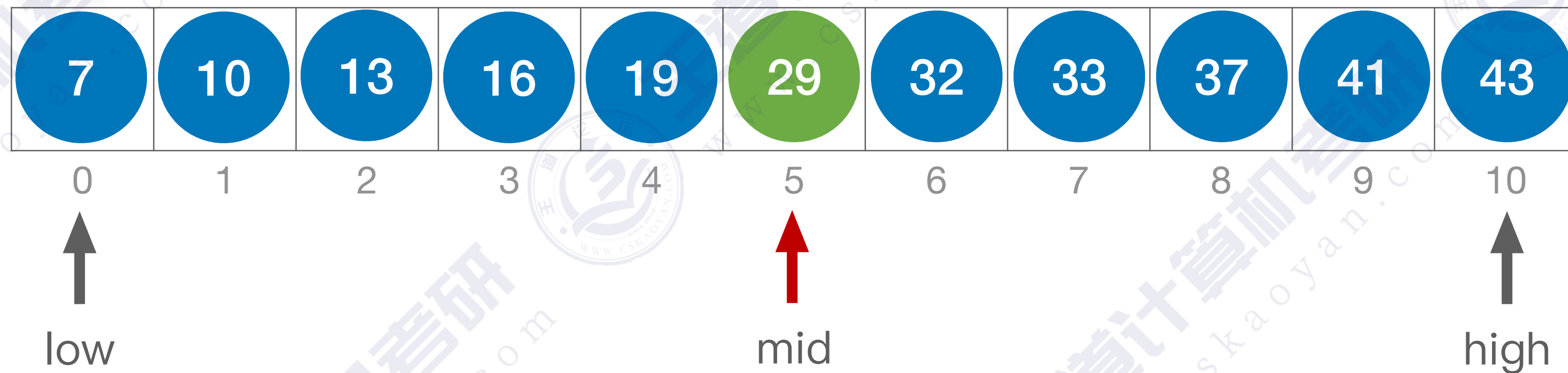


别瞎说啊

拓展思考



如果 $mid = \lceil (low + high)/2 \rceil$
辣么，判定树是什么亚子？



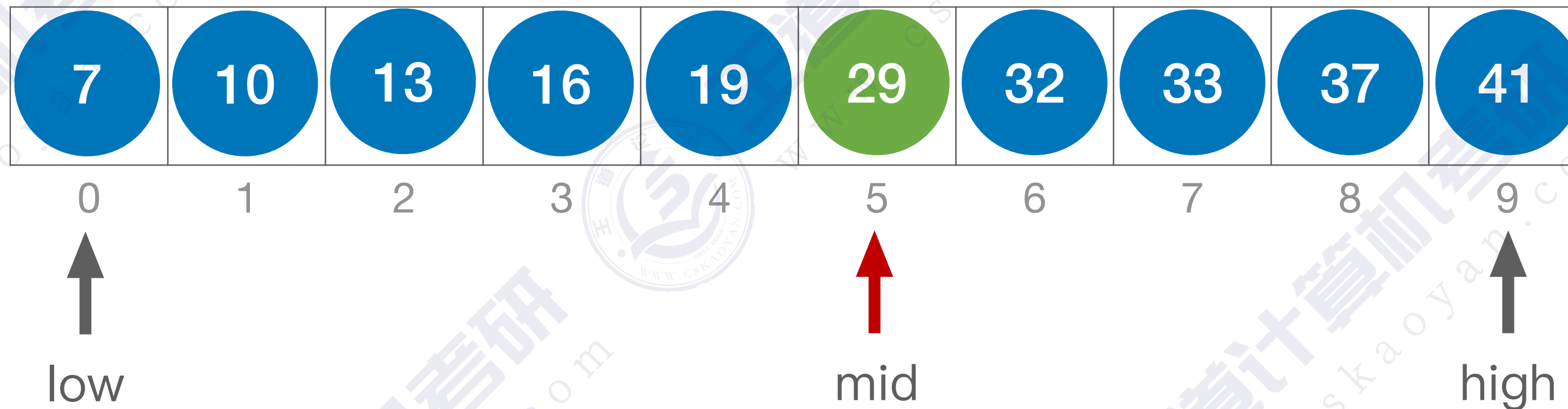
$$mid = \lceil (low + high)/2 \rceil$$

如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

拓展思考



如果 $mid = \lceil (low + high)/2 \rceil$
辣么，判定树是什么亚子？

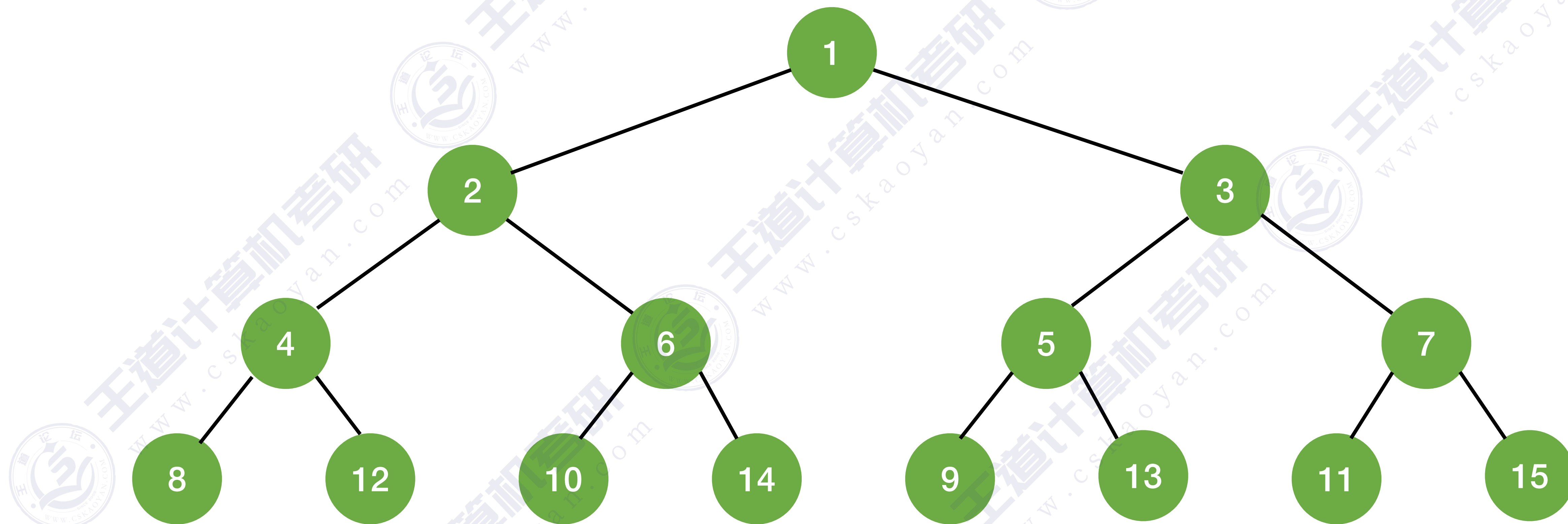


$$mid = \lceil (low + high)/2 \rceil$$

如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分 **多** 一个元素

拓展思考

折半查找的判定树中，若 $mid = \lceil (low + high)/2 \rceil$ ，则对于任何一个结点，必有：
左子树结点数-右子树结点数=0或1



注：图中数字只是一个编号，并不是关键字的值

本节内容

分块查找

知识总览



算法思想

分块查找



查找效率分析 (ASL)

分块查找的算法思想

“索引表”中保存每个分块的最大关键字和分块的存储区间

10	20	30	40	50
[0,1]	[2,5]	[6,8]	[9,10]	[11,13]

0 1 2 3 4

7	10	13	19	16	20	27	22	30	40	36	43	50	48	...
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

//索引表

```
typedef struct {  
    ElemType maxVal;  
    int low,high;  
}Index;
```

//顺序表存储实际元素

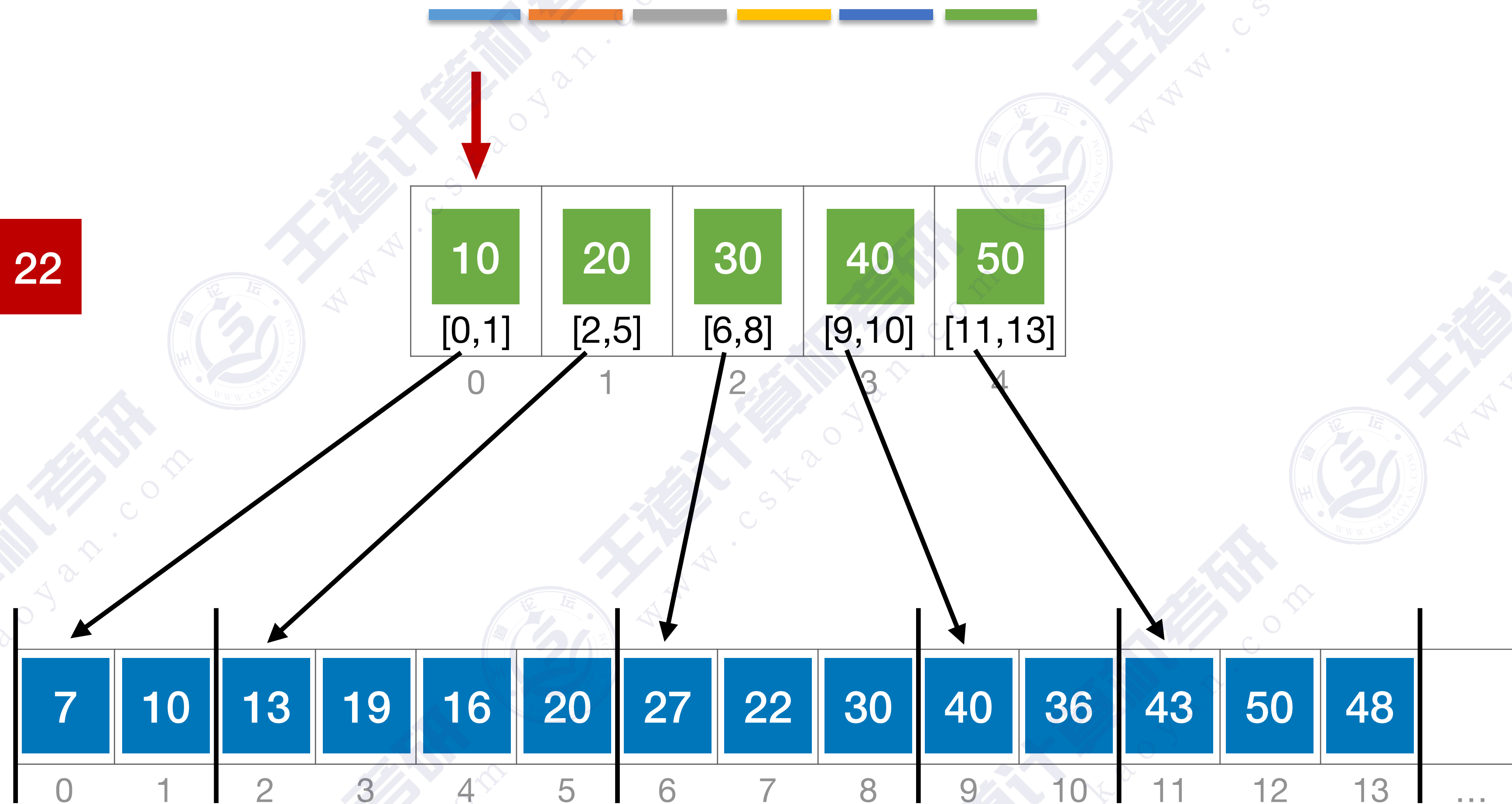
```
ElemType List[100];
```

特点：块内无序、块间有序

分块查找的算法思想

查找目标:

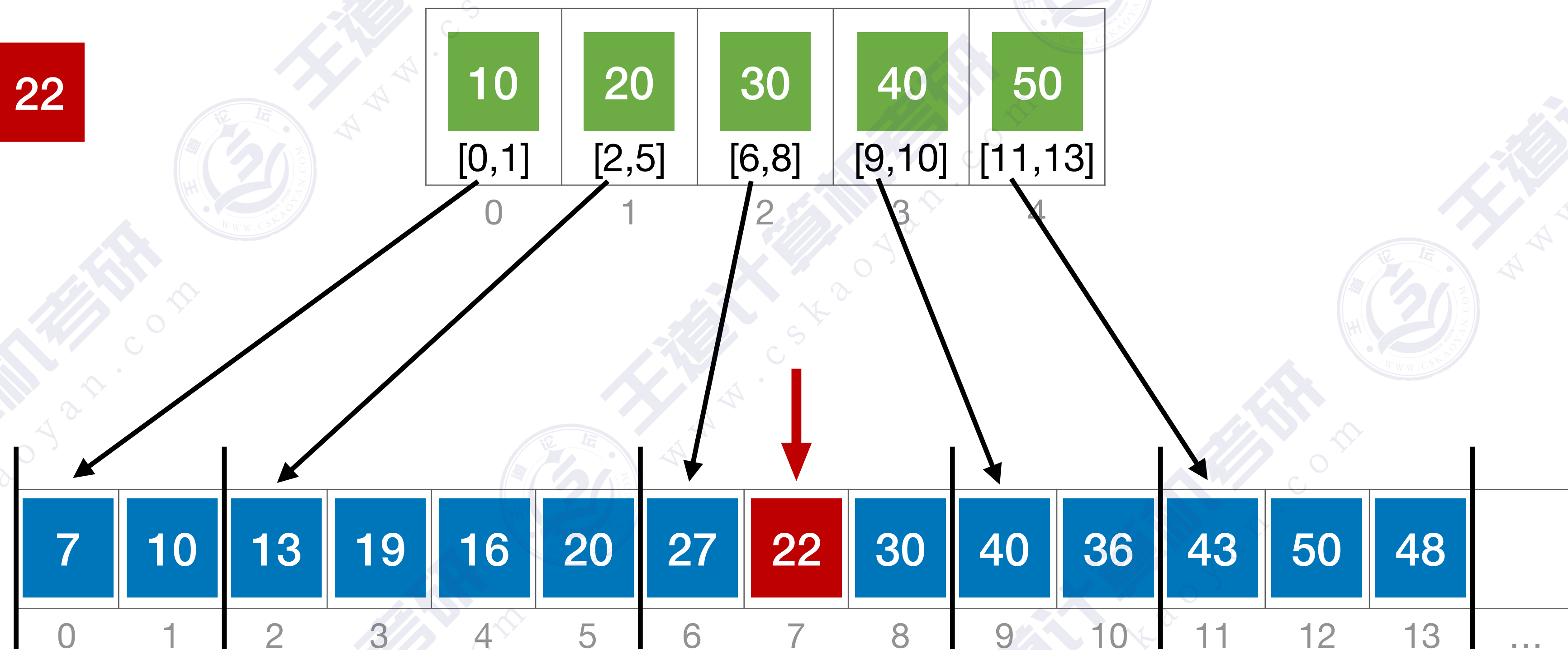
22



分块查找的算法思想

查找目标:

22

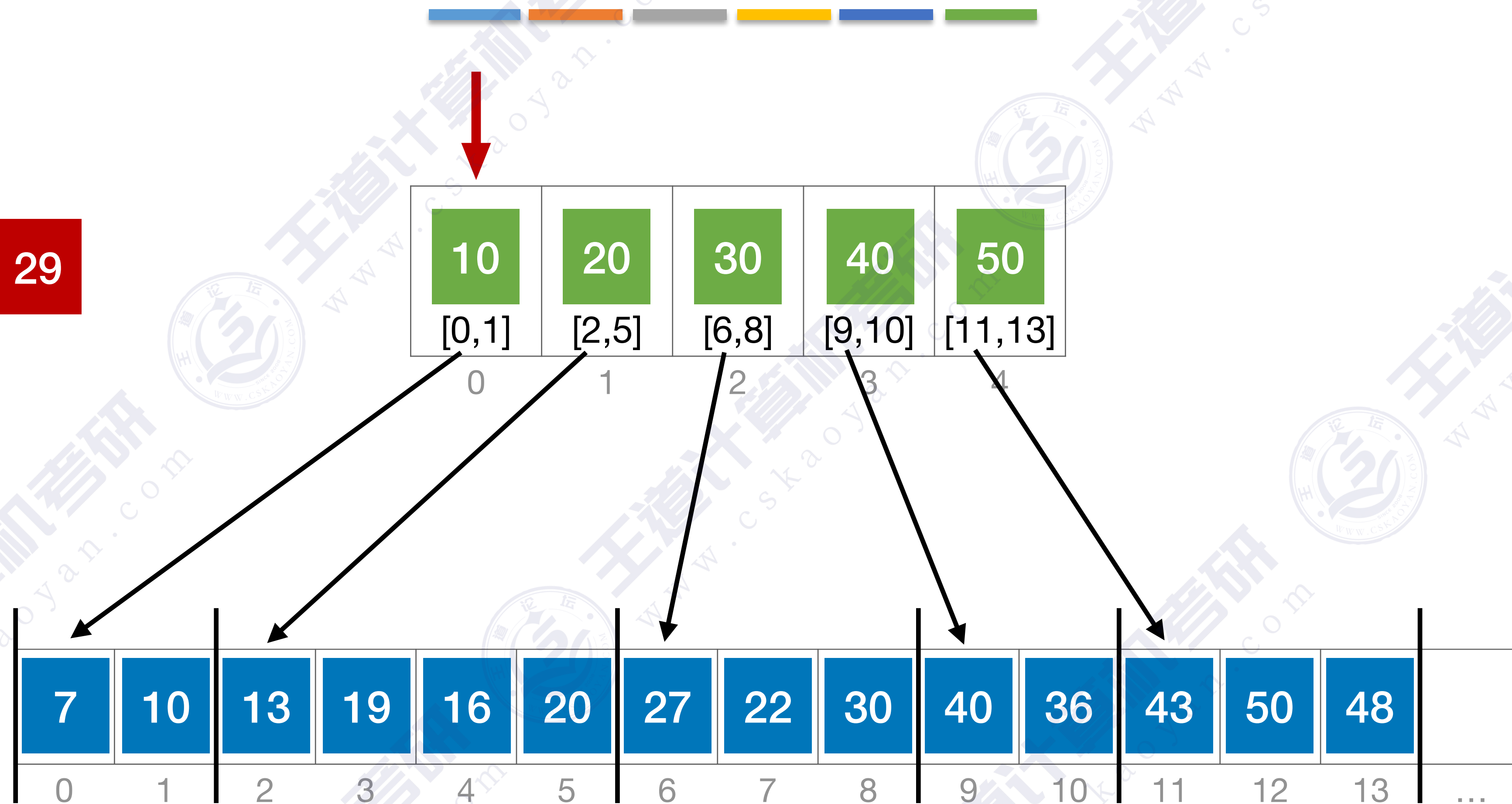


查找成功

分块查找的算法思想

查找目标:

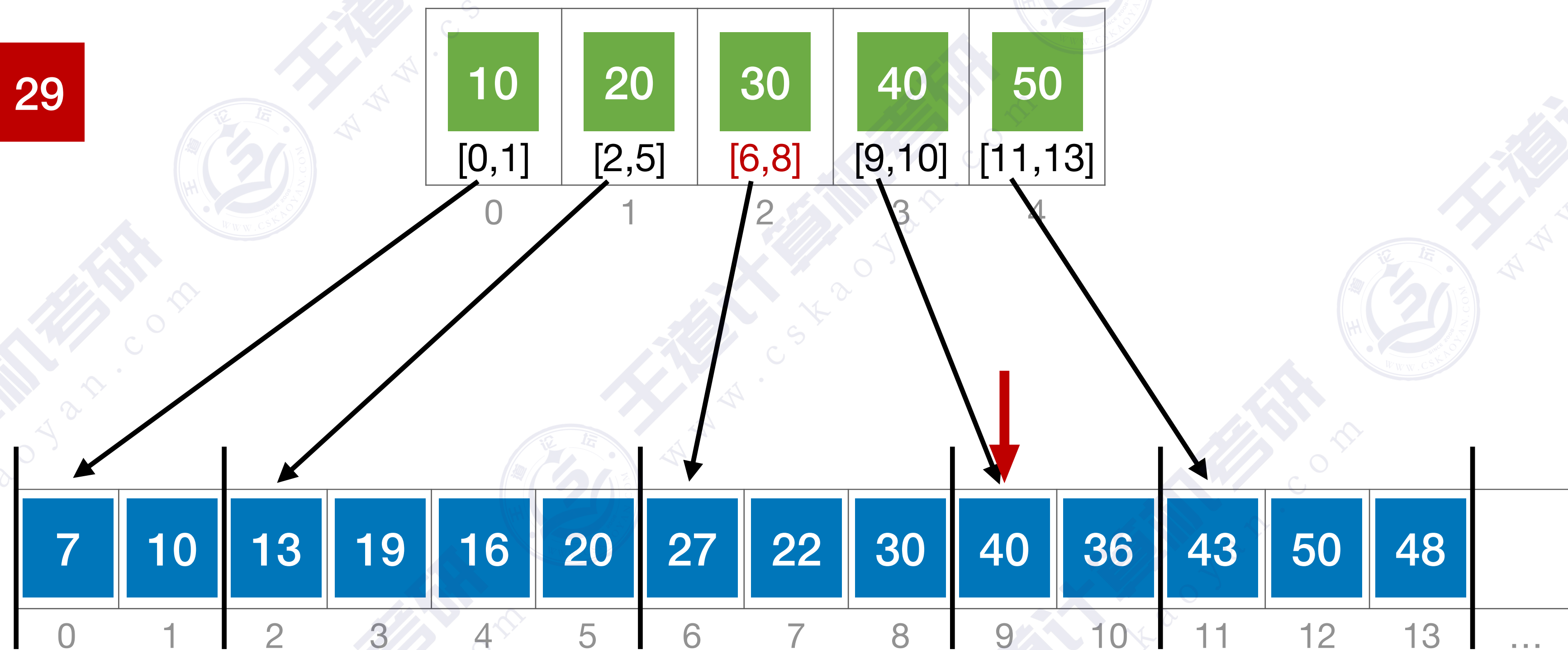
29



分块查找的算法思想

查找目标:

29



超出分块范围，查找失败

分块查找的算法思想

“索引表”中保存每个分块的最大关键字和分块的存储区间

10	20	30	40	50
[0,1]	[2,5]	[6,8]	[9,10]	[11,13]

0 1 2 3 4

7	10	13	19	16	20	27	22	30	40	36	43	50	48	...
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

//索引表

```
typedef struct {  
    ElemType maxVal;  
    int low,high;  
}Index;
```

//顺序表存储实际元素

```
ElemType List[100];
```

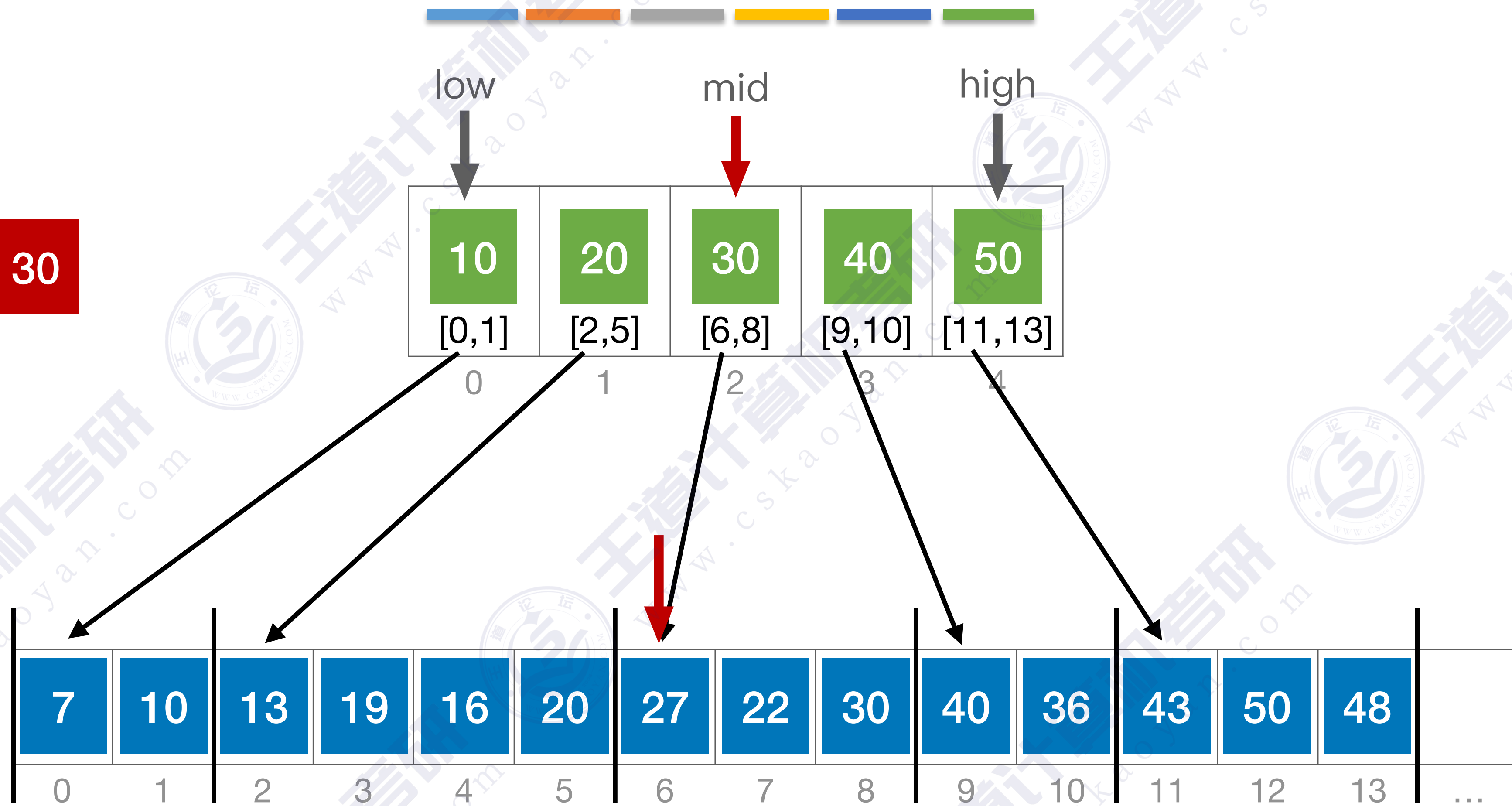
分块查找，又称索引顺序查找，算法过程如下：

- ①在索引表中确定待查记录所属的分块（可顺序、可折半）
- ②在块内顺序查找

用折半查找索引

查找目标:

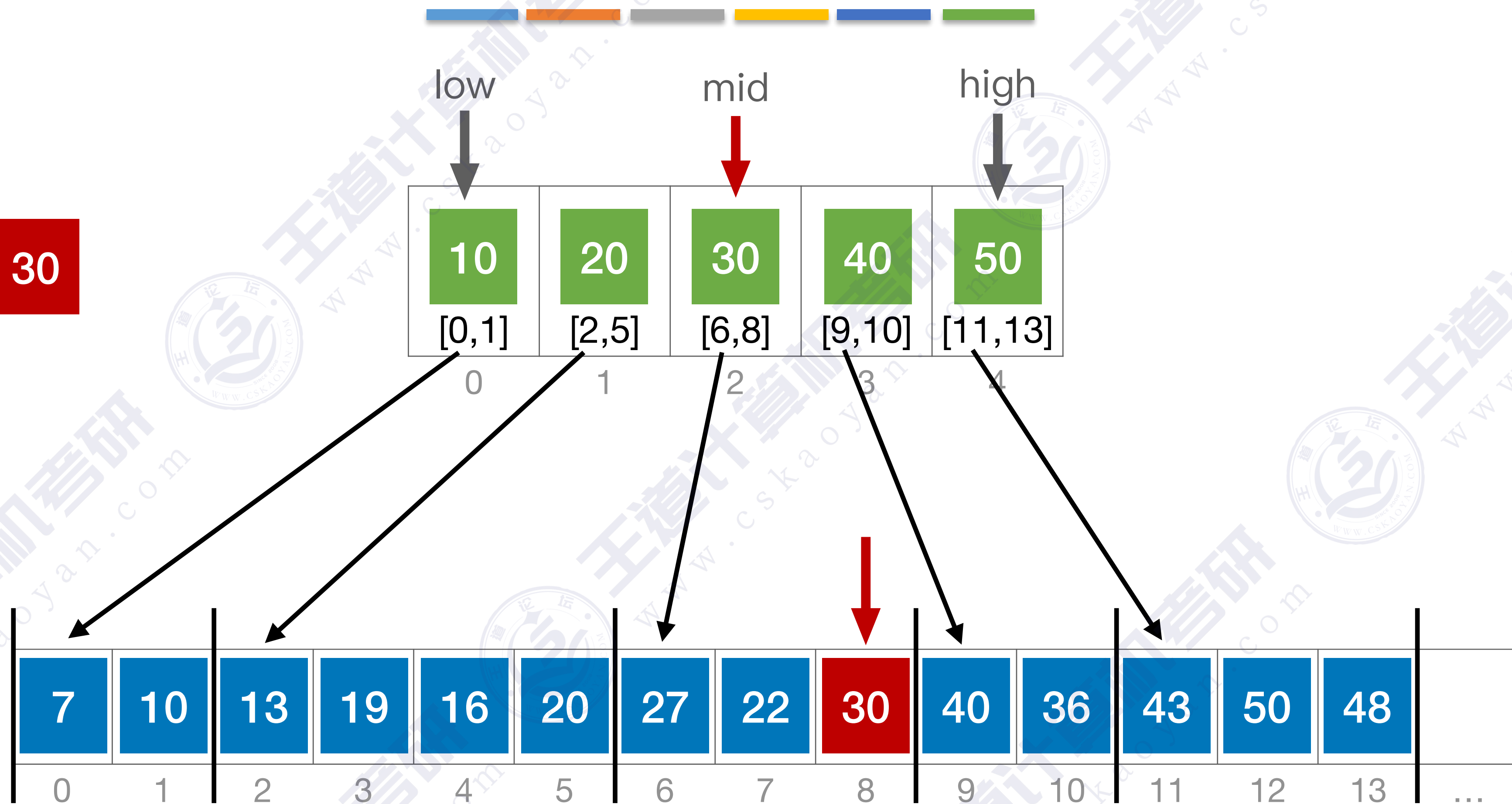
30



用折半查找查索引

查找目标:

30

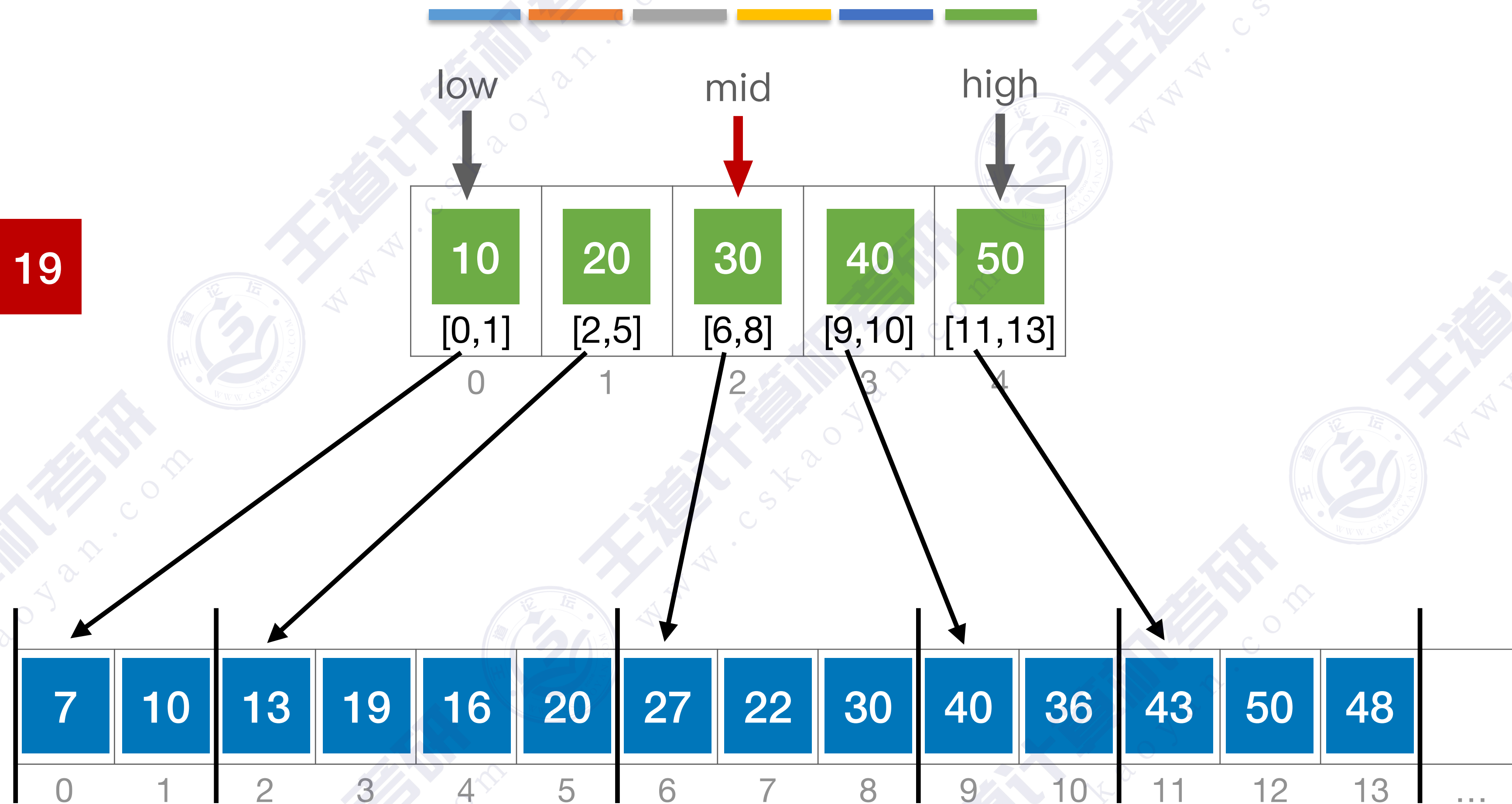


查找成功

用折半查找查索引

查找目标:

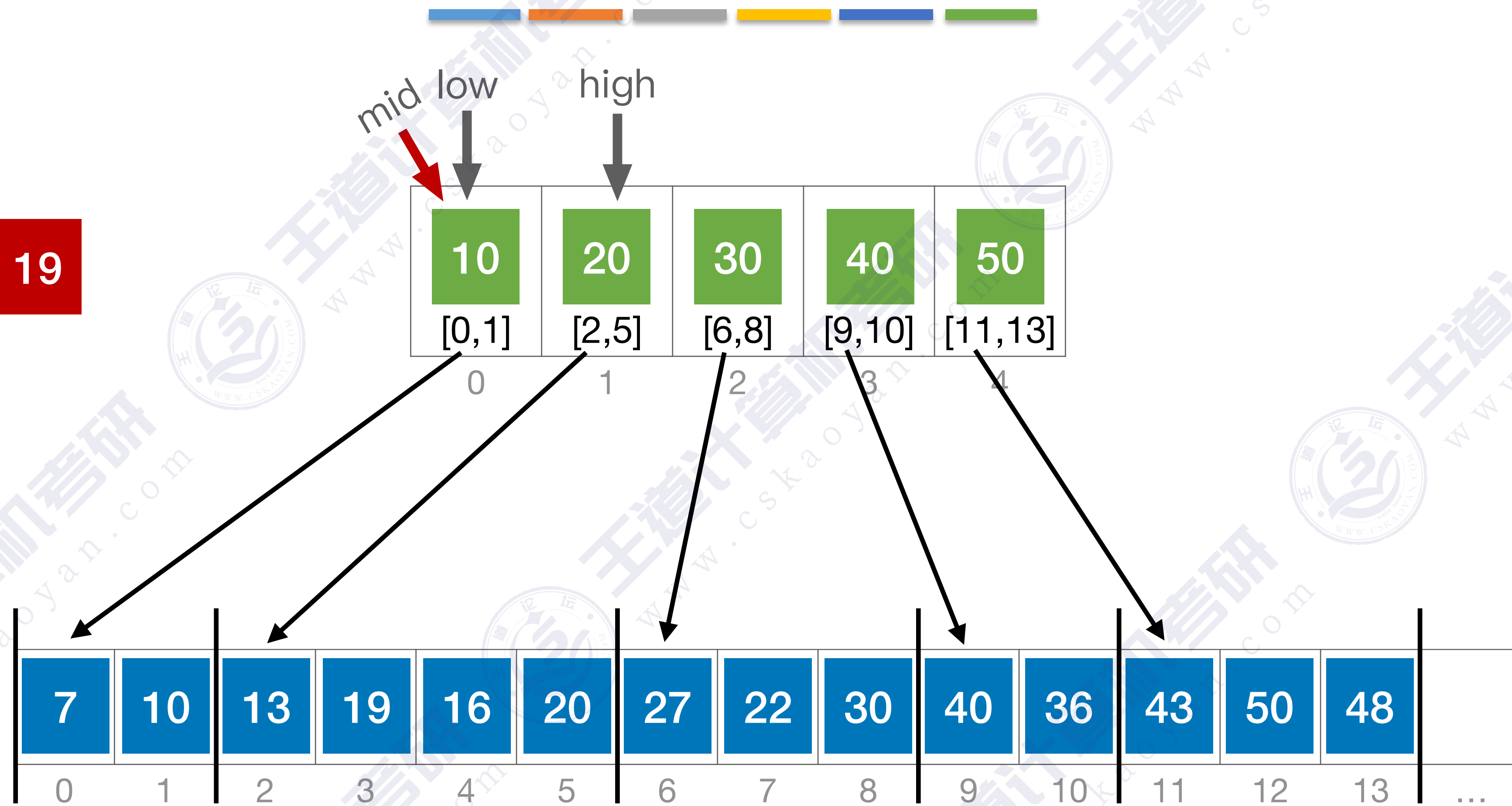
19



用折半查找查索引

查找目标:

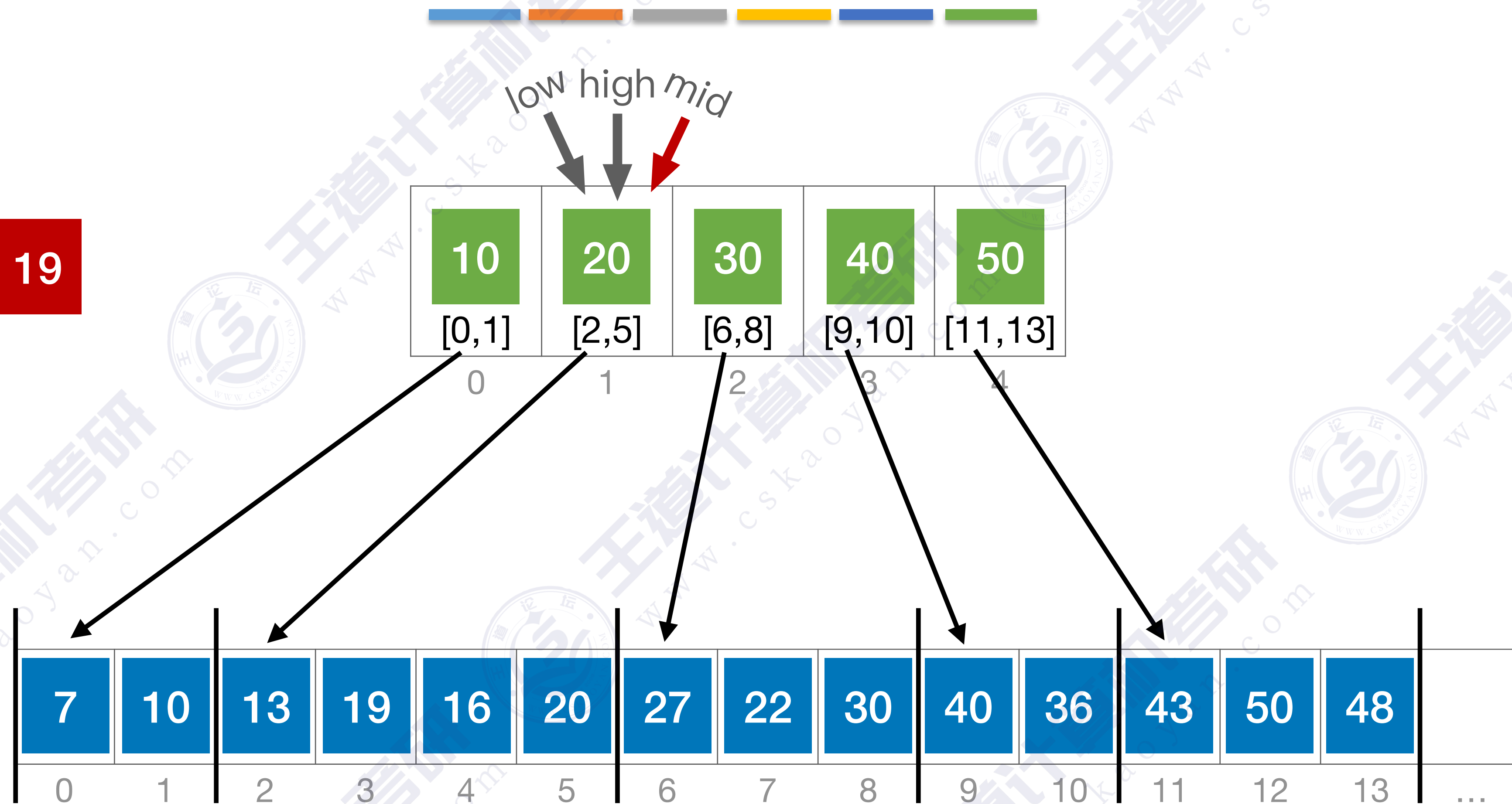
19



用折半查找索引

查找目标:

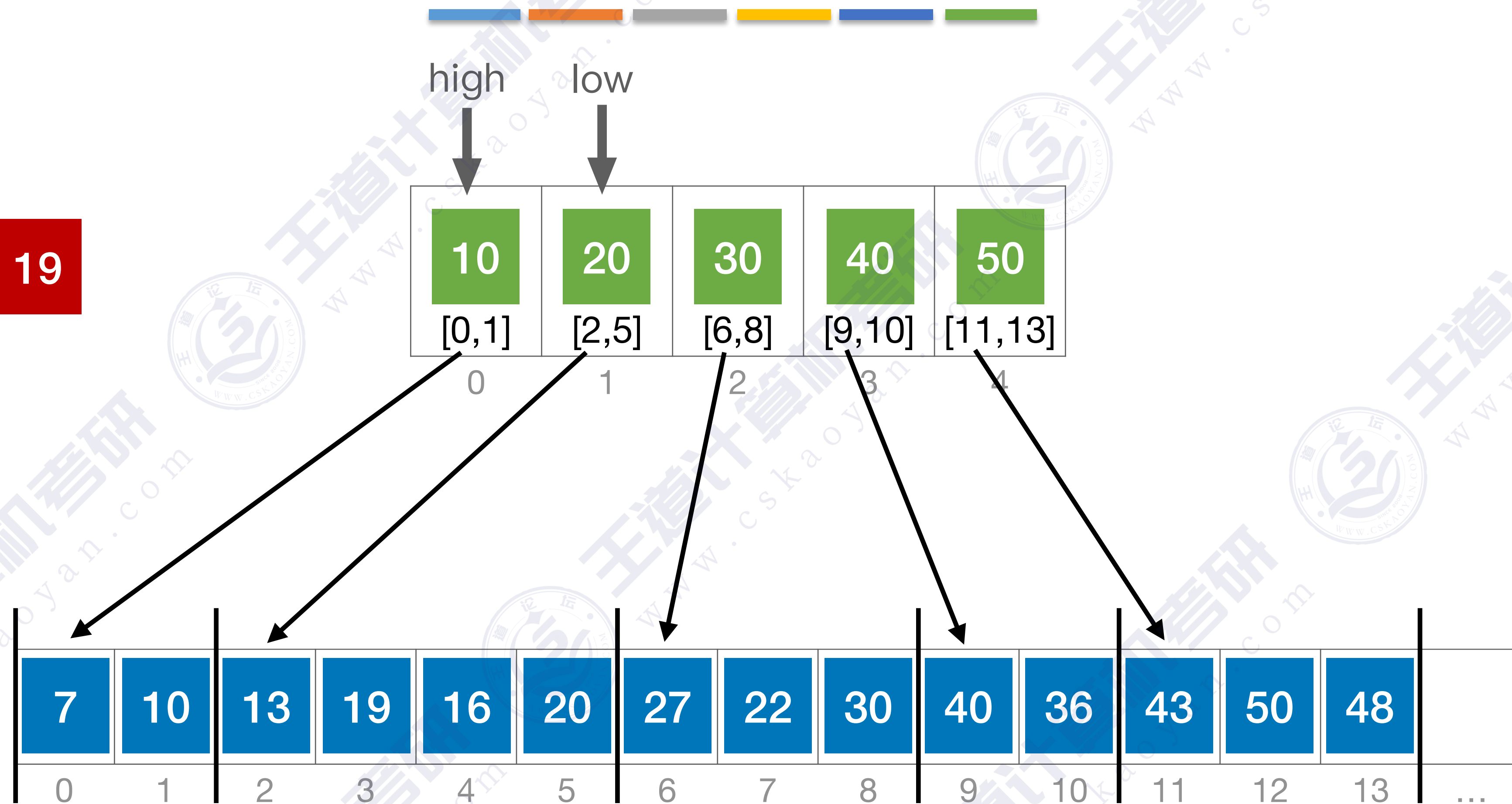
19



用折半查找查索引

查找目标:

19

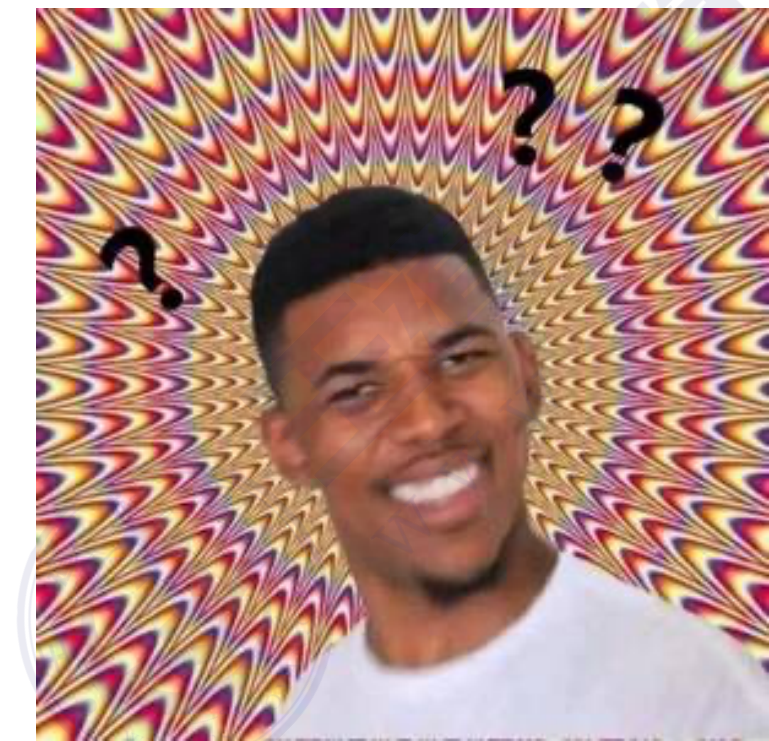
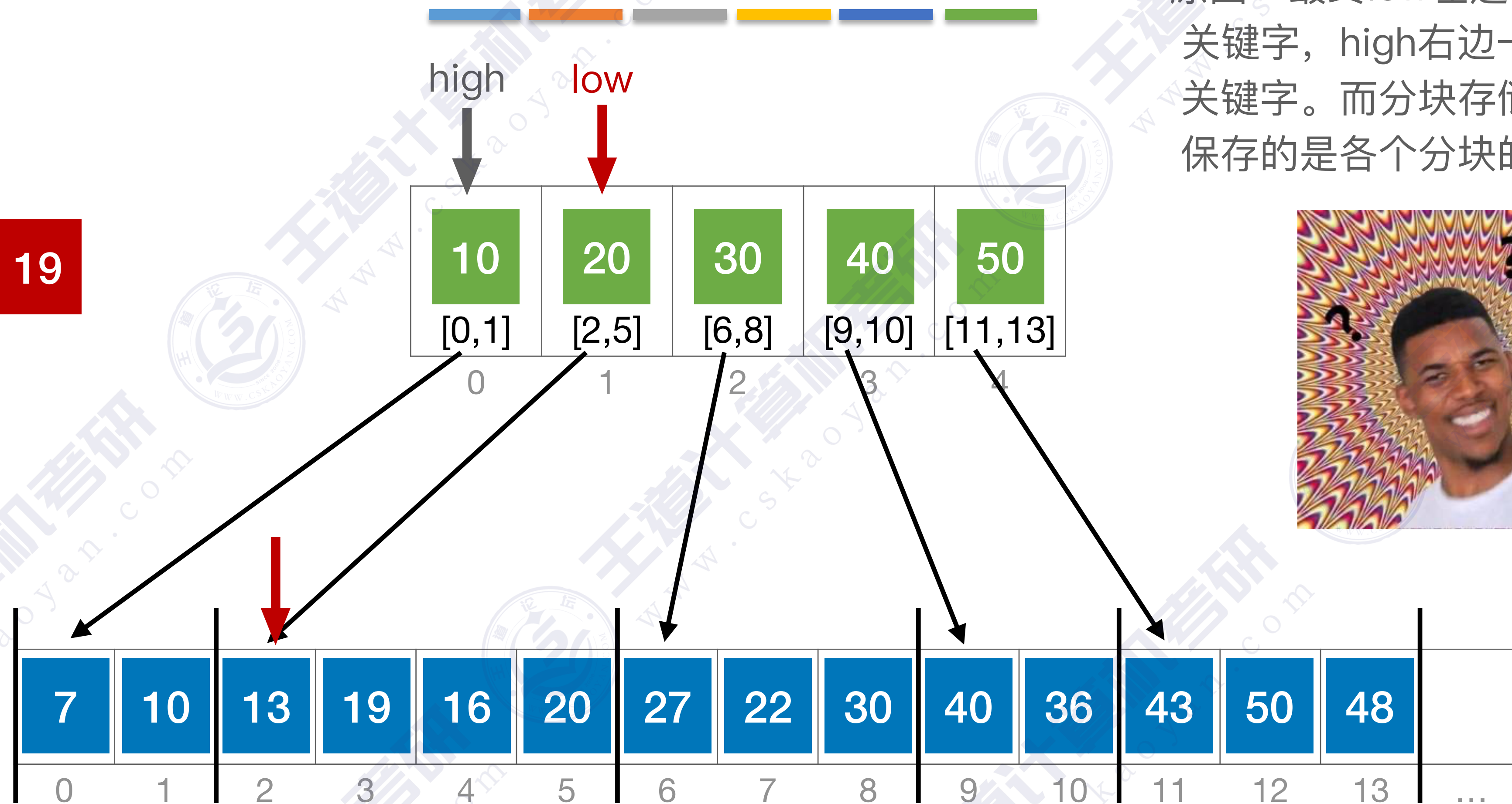


用折半查找索引

原因：最终low左边一定小于目标关键字，high右边一定大于目标关键字。而分块存储的索引表中保存的是各个分块的最大关键字

查找目标：

19

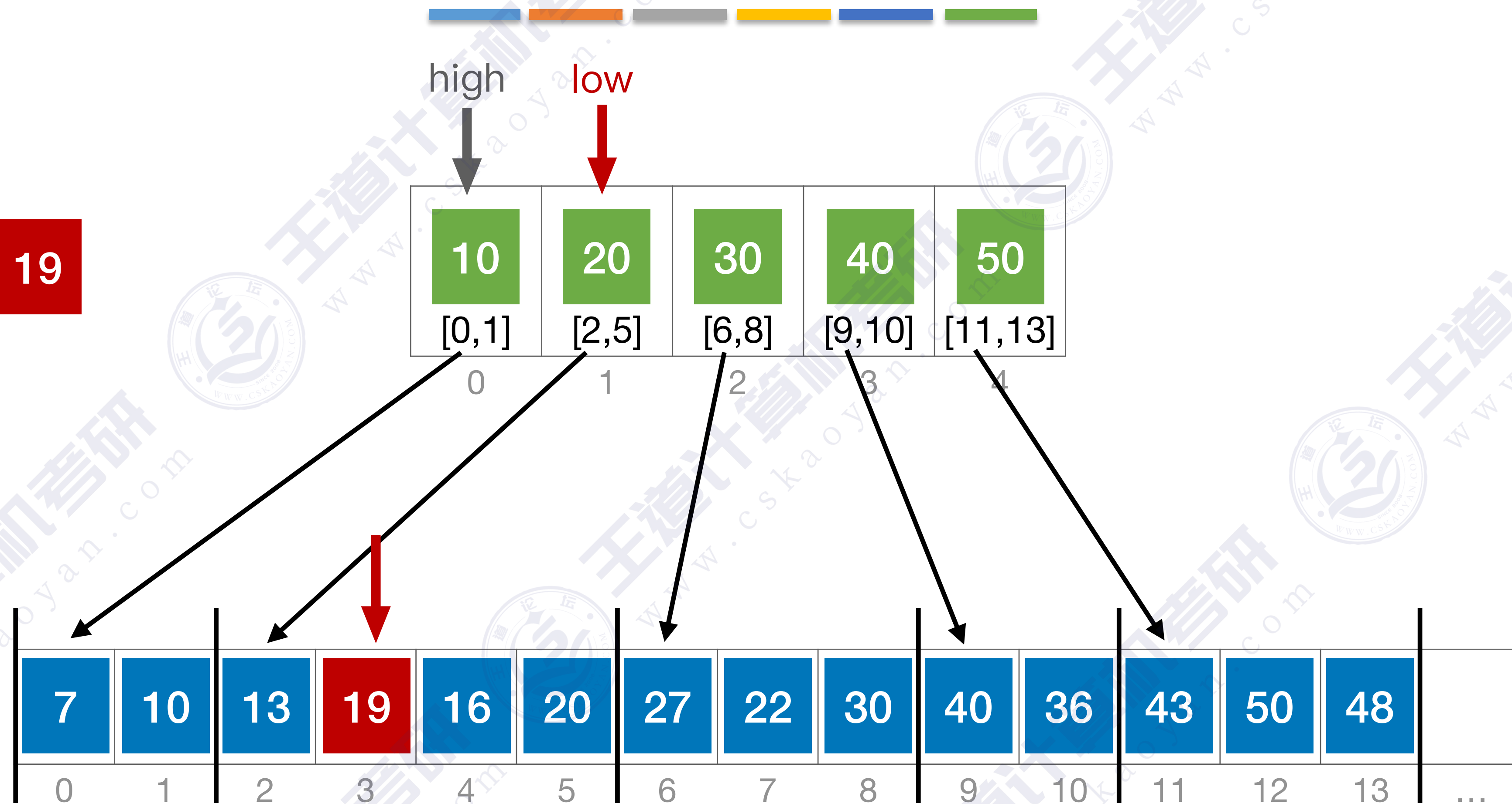


若索引表中不包含目标关键字，则折半查找索引表最终停在 $low > high$ ，要在low所指分块中查找

用折半查找查索引

查找目标:

19

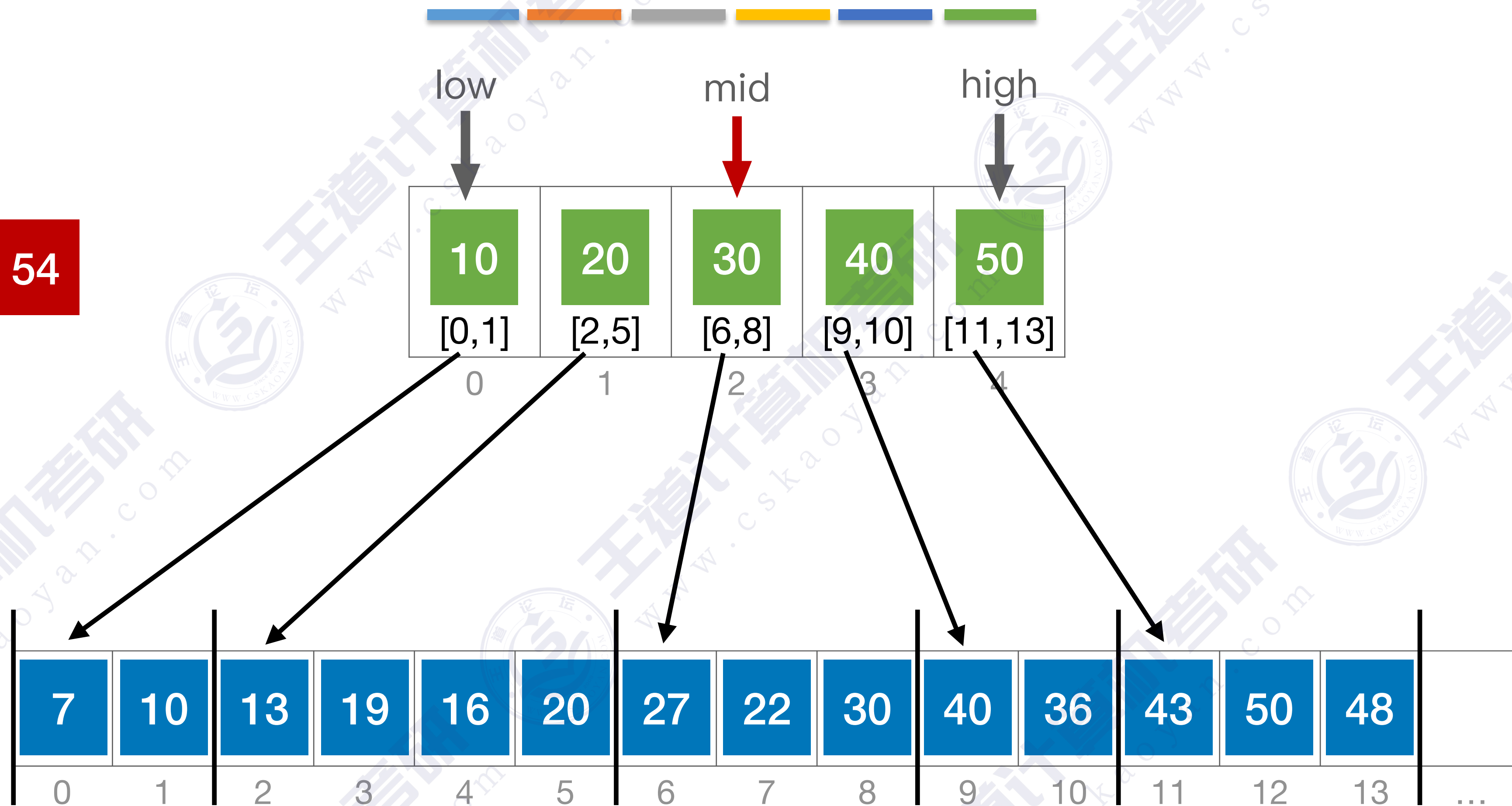


查找成功

用折半查找查索引

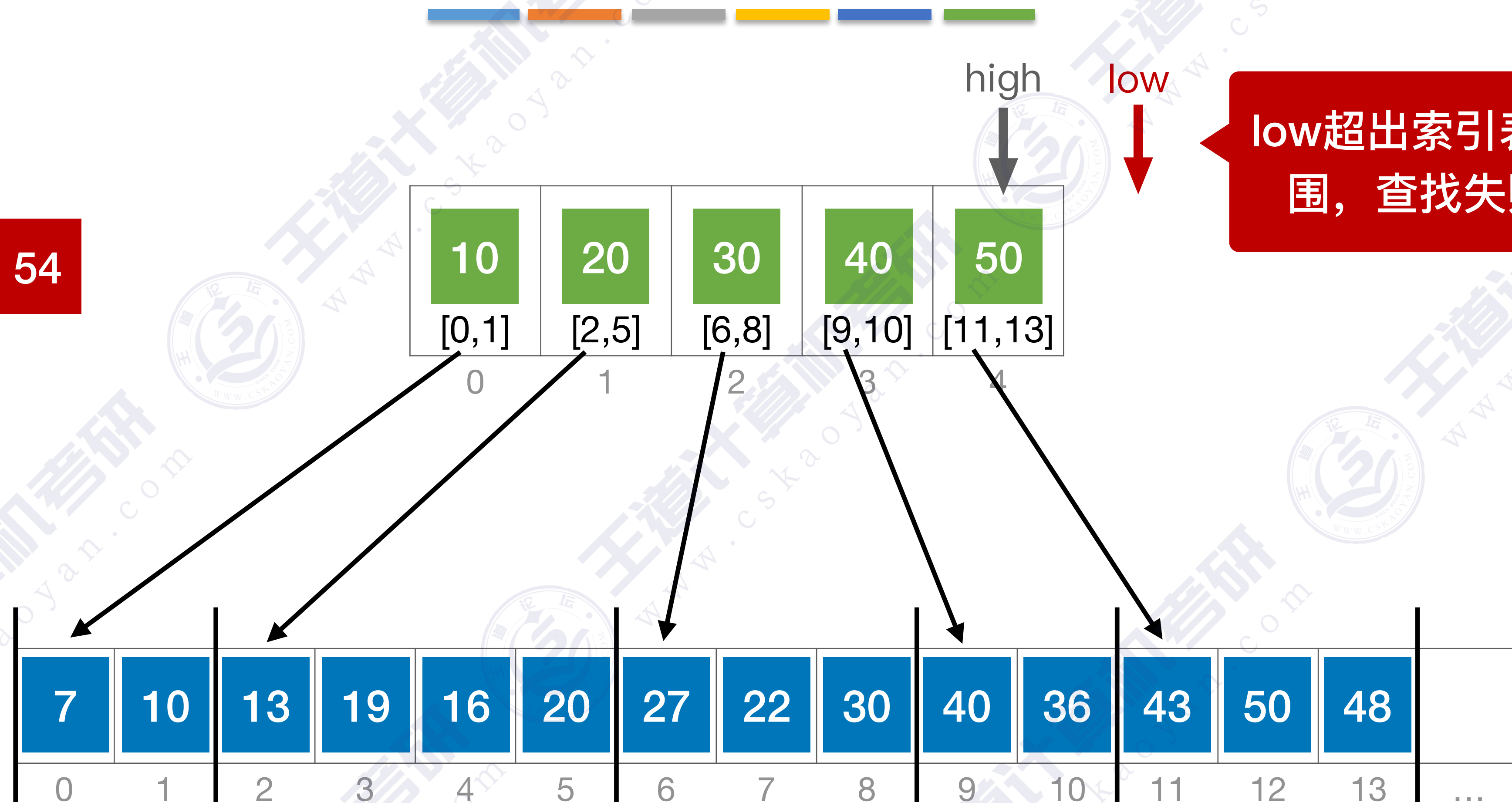
查找目标:

54



用折半查找索引

查找目标: **54**

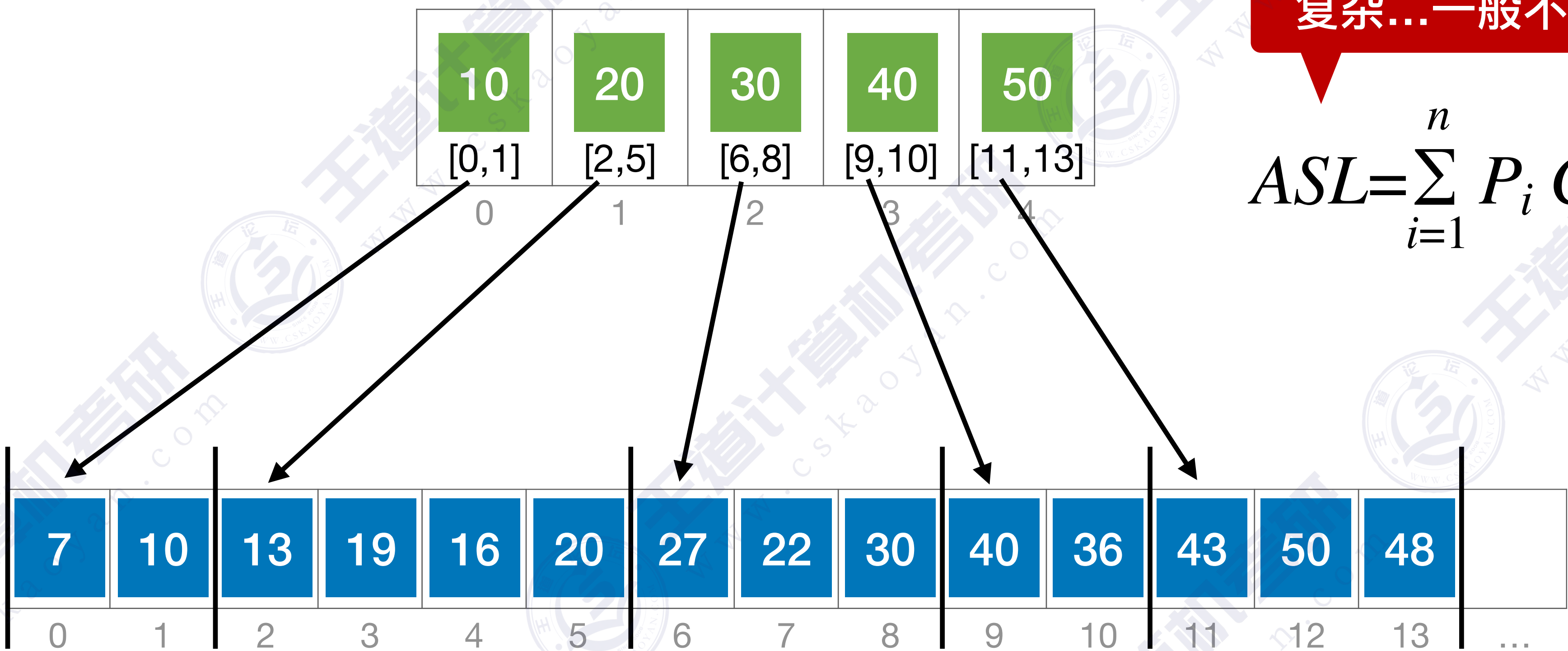


若索引表中不包含目标关键字, 则折半查找索引表最终停在 $low > high$, 要在low所指分块中查找

查找效率分析 (ASL)

查找失败的情况更复杂...一般不考

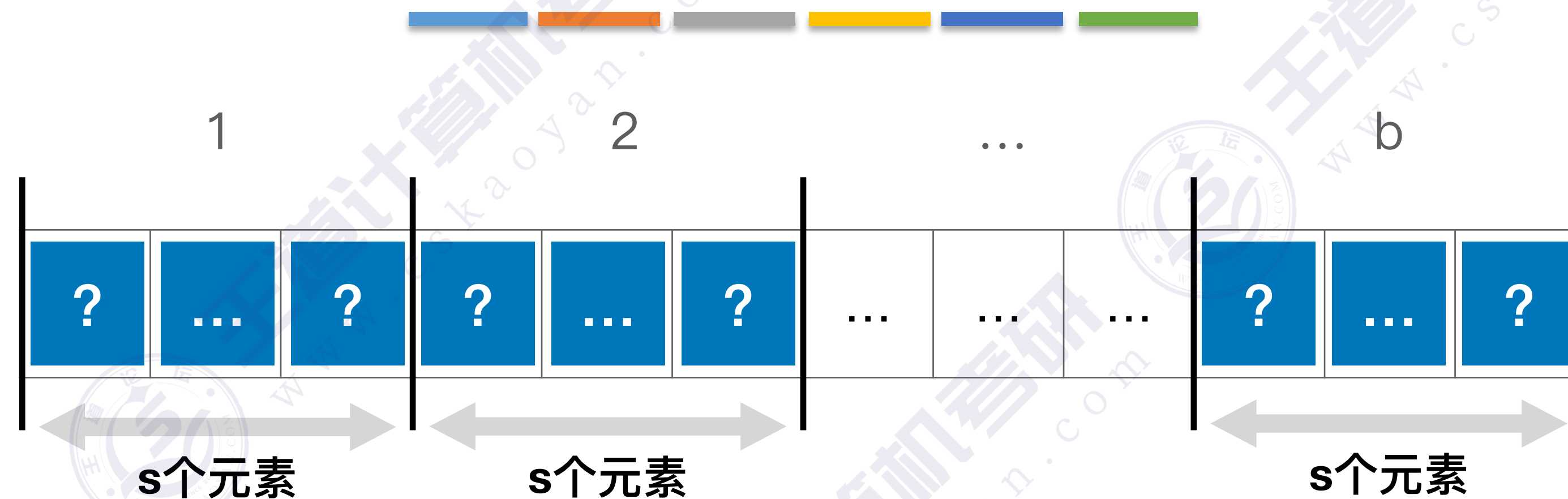
$$ASL = \sum_{i=1}^n P_i C_i$$



共有14个元素，各自被查概率为1/14
若索引表采用顺序查找，则 7：2次、10：3次、13：3次...
若索引表采用折半查找，则30：4次、27：2次？



查找效率分析 (ASL)



假设，长度为 n 的查找表被均匀地分为 b 块，每块 s 个元素

设索引查找和块内查找的平均查找长度分别为 L_I 、 L_S ，则分块查找的平均查找长度为

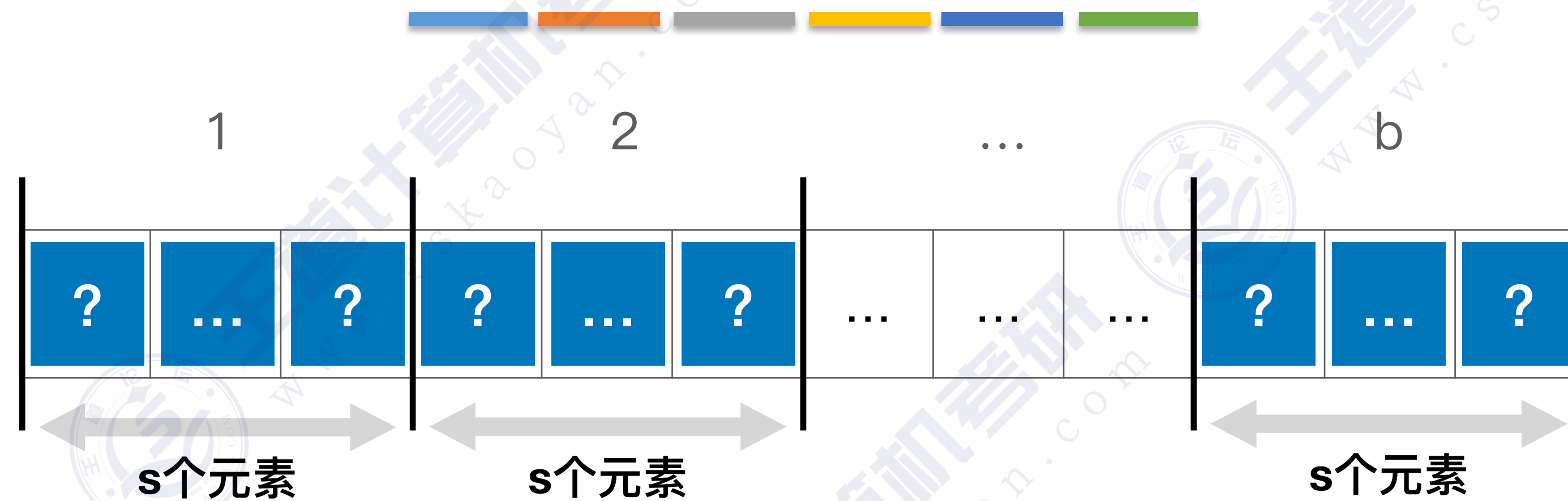
$$ASL = L_I + L_S$$

用顺序查找查索引表，则 $L_I = \frac{(1 + 2 + \dots + b)}{b} = \frac{b + 1}{2}$, $L_S = \frac{(1 + 2 + \dots + s)}{s} = \frac{s + 1}{2}$

$$\text{则 } ASL = \frac{b + 1}{2} + \frac{s + 1}{2} = \frac{s^2 + 2s + n}{2s}, \text{ 当 } s = \sqrt{n} \text{ 时, } ASL_{\text{最小}} = \sqrt{n} + 1$$

若 $n=10000$ ，则
 $ASL_{\min}=101$

查找效率分析 (ASL)



假设，长度为 n 的查找表被均匀地分为 b 块，每块 s 个元素

设索引查找和块内查找的平均查找长度分别为 L_I 、 L_S ，则分块查找的平均查找长度为

$$ASL = L_I + L_S$$

用折半查找查索引表，则 $L_I = \lceil \log_2(b+1) \rceil$ ， $L_S = \frac{(1+2+\dots+s)}{s} = \frac{s+1}{2}$

$$\text{则 } ASL = \lceil \log_2(b+1) \rceil + \frac{s+1}{2}$$

知识回顾与重要考点

又称“索引顺序查找”，数据分块存储，块内无序、块间有序

算法思想

索引表中记录每个分块的最大关键字、分块的区间

先查索引表（顺序或折半）、再对分块内进行顺序查找

ASL=查索引表的平均查找长度+查分块的平均查找长度

ASL

设n个记录，
均匀分为b块，
每块s个记录

顺序查找索引表

$$ASL = \frac{b+1}{2} + \frac{s+1}{2}$$

当 $s=\sqrt{n}$ 时， $ASL_{\text{最小}}=\sqrt{n}+1$

折半查找索引表

$$ASL = [\log_2(b+1)] + \frac{s+1}{2}$$

易错点

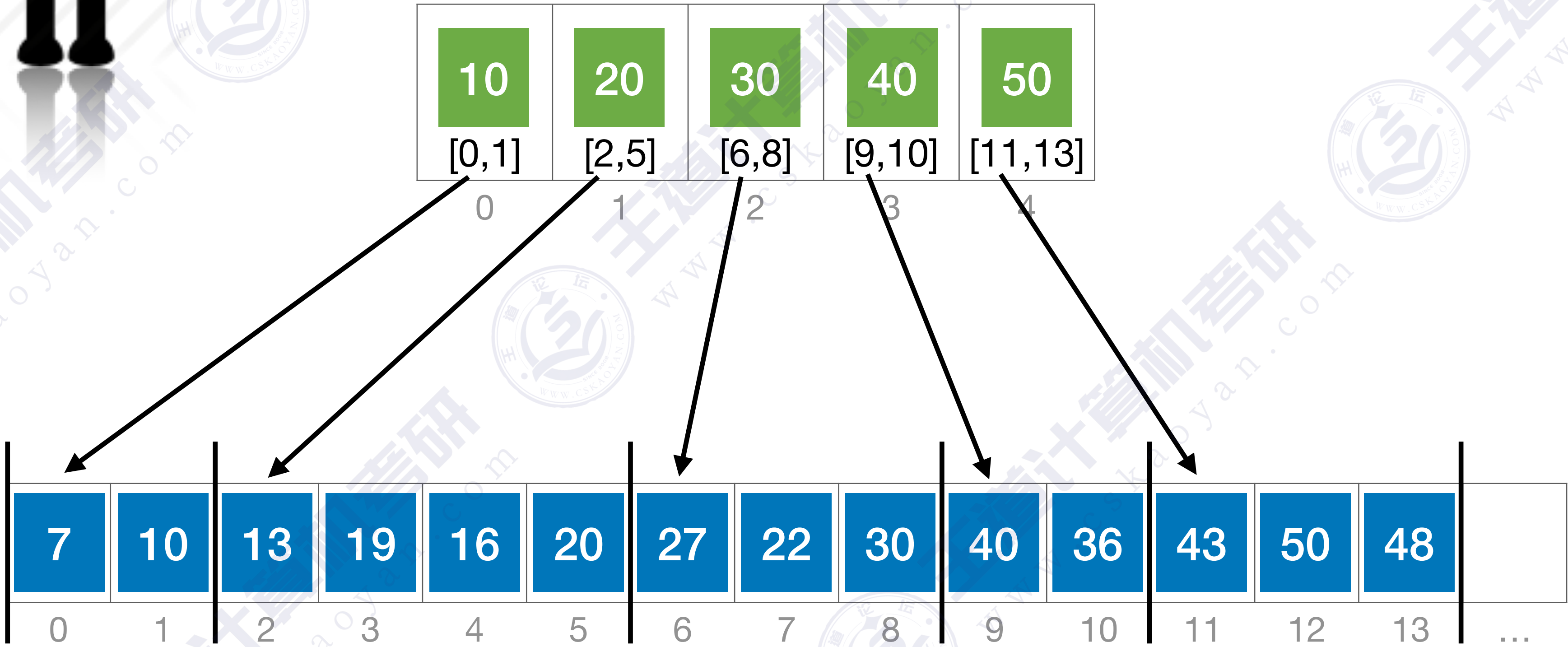
对索引表进行折半查找时，若索引表中不包含目标关键字，
则折半查找最终停在 $low>high$ ，要在 low 所指分块中查找

分块查找

拓展思考



若查找表是“动态查找表”，有木有更好的实现方式？



拓展思考



若查找表是“动态查找表”，有木有更好的实现方式？——链式存储

